

FINAL YEAR PROJECT REPORT

Project title:

**Design and development of a multi-agent system for queue
waiting-time estimation from video streams**

Prepared by:

Salmen Ben Ammar

Host Company:



Company Supervisor:

Ramzi Guesmi

Academic Supervisor:

Tarek Hamrouni

Academic Year 2025–2026



Dedications

To my parents,

No dedication could ever truly reflect my unending love and gratitude for all you've done for my education and well-being. May you recognize in this simple work the fulfillment of your dreams and the reward of your many sacrifices.

To my sister,

Your affection, support, and constant encouragement have been an immense source of motivation throughout my journey. Your belief in me helped me overcome challenges and stay focused on my goals. I wish you a life filled with success and happiness.

To my family,

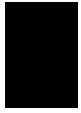
Your presence and unconditional love are my greatest treasures. Thank you for everything. You will forever remain my inexhaustible source of support, love, and comfort.

To all my friends,

My second family who made my journey an indescribable experience, and who have always encouraged me and believed in me. I wish you a future filled with achievement and joy.

To all of you

I dedicate this work.



Acknowledgements

At the conclusion of this project, I wish to express my sincere gratitude to the individuals who have greatly contributed to its successful completion.

First, I would like to express my profound appreciation to the entire team at **Data-DoIt** for graciously providing me the opportunity to carry out this final-year project.

I would also like to extend my sincere thanks to my professional supervisor, **Mr. Ramzi Guesmi**, for his unwavering support, constant availability, patience, and his relevant and constructive guidance, which proved invaluable throughout this project.

I warmly thank my academic supervisor, **Mr. Tarek Hamrouni**, for his constant availability, patience, and insightful advice, which were essential for the successful completion of this work. His guidance, encouragement, and constructive feedback not only helped me overcome numerous challenges but also inspired me to strive for excellence and deepen my understanding throughout this project.

I would also like to take this opportunity to express my gratitude to the entire faculty of **Sesame University**, whose contributions have been fundamental to my education over the past three years.

Finally, I sincerely thank the members of the jury for agreeing to evaluate my work. Their expertise and commitment are greatly appreciated, and I wish to express my deep gratitude for their contribution to the assessment of this project.



Table of Content

List of Figures	ix
List of Tables	xi
General Introduction	xii
1 Context of the Project	1
1.1 Introduction	2
1.2 Host Organization	2
1.2.1 DataDoIt	2
1.2.2 Sectors of Activity	3
1.3 Presentation of the Project	4
1.3.1 Project Context	4
1.3.2 Problem Statement	4
1.3.3 Study of Existing Approaches	5
1.3.3.1 Queue Management Platforms	6
1.3.3.2 Sensor-Based Analytics	6
1.3.3.3 Vision-Based Monitoring Frameworks	6
1.3.3.4 Positioning of the Proposed Solution	7
1.3.4 Proposed Solution	7
1.3.5 Objectives of the Project	8
1.3.6 Project Scope and Boundaries	9
1.3.7 Target Audience	9
1.4 Project Management Methodology	9

1.4.1	Choice of Methodology: Agile	9
1.4.2	SCRUM	10
1.4.2.1	Module A: Backend Core (Sprints 1–2)	10
1.4.2.2	Module B: Web Experience (Sprint 3)	10
1.4.2.3	Module C: Automation and Finalization (Sprint 4)	11
1.4.3	Sprint Planning	11
1.5	Conclusion	13
2	Requirements Specification and Analysis	14
2.1	Introduction	15
2.2	Requirements Specification	15
2.2.1	Identification of Actors	15
2.2.2	Authentication and Access Governance	16
2.2.3	Functional Requirements	16
2.2.4	Non-functional Requirements	17
2.3	Product Backlog	18
2.4	Global Use Case Diagram	19
2.5	Sprint Scope Overview	21
2.6	Project Environment	21
2.6.1	Hardware and Software Environment	21
2.6.2	Development Tools	21
2.6.3	Technologies Used	23
2.7	Software Architecture	23
2.7.1	Overall Architecture	23
2.7.2	Backend Architecture	24
2.7.3	Frontend Architecture	25
2.8	Conclusion	25

3	Sprint 1: Perception and Queue Analytics	26
3.1	Introduction	27
3.2	Sprint Objectives	27
3.3	Sprint Backlog	28
3.4	Sprint Analysis	29
3.4.1	Runtime Scenarios	29
3.5	Design	30
3.5.1	Activity Diagram	30
3.5.2	Sequence Diagram	31
3.5.3	Operational Scenarios	31
3.6	Implementation	32
3.6.1	Detection Layer	32
3.6.2	Tracking Layer	32
3.6.3	Zone-Aware Filtering	32
3.6.4	Main Loop Integration	32
3.6.5	Queue Event Windows and Rate Estimation	32
3.6.6	Wait-Time Logic and Stability Handling	33
3.6.7	Runtime Metric Contract	33
3.7	Conclusion	34
3.8	Sprint Retrospective	34
4	Sprint 2: Backend Services and API Integration	35
4.1	Introduction	36
4.2	Sprint Objectives	36
4.3	Sprint Backlog	36
4.4	Design	37
4.4.1	Sequence Diagrams	37

4.4.2	Backend Class Diagram	37
4.5	Implementation	38
4.5.1	Camera Source Integration	38
4.5.2	Authenticated API Access	39
4.5.3	Camera Discovery and Stream Setup	39
4.5.4	API Layer and Data Contract	39
4.5.5	Live Video Delivery	39
4.5.6	Runtime Control and Live Events	40
4.5.7	Backend Persistence Architecture	40
4.5.8	Persistence and Recovery	40
4.6	Conclusion	40
4.7	Sprint Retrospective	41
5	Sprint 3: Web Dashboard and Live Monitoring	42
5.1	Introduction	43
5.2	Sprint Objectives	43
5.3	Sprint Backlog	43
5.4	Design	45
5.4.1	UI and UX Design	45
5.4.2	Sequence Diagrams	45
5.4.3	Design Rationale	46
5.5	Implementation	46
5.5.1	Project Setup and Layout Skeleton	46
5.5.2	Styling and Reusable Interface Rules	46
5.5.3	API Integration	46
5.5.4	Authentication and Role-Based Navigation	47
5.5.5	Dashboard State Orchestration	48

5.5.6	Live Update Reliability	49
5.5.7	Live Video Delivery	49
5.5.8	Dashboard UI Integration	50
5.5.9	Integration Testing	55
5.6	Conclusion	56
5.7	Sprint Retrospective	56
6	Sprint 4: Automation, Evaluation, and Optimization	57
6.1	Introduction	59
6.2	Sprint Objectives	59
6.3	Sprint Backlog	59
6.4	Design	60
6.4.1	Alert Workflow Diagram	60
6.4.2	Verification Plan	61
6.4.3	Performance Targets	61
6.5	Implementation	61
6.5.1	Alert Payload Structure	61
6.5.2	Delivery Reliability	62
6.5.3	Telegram Bot Setup	62
6.5.4	Alert Dispatch in the Runtime Loop	62
6.5.5	n8n Workflow Routing and Telegram Delivery	62
6.5.6	Workflow Validation	63
6.5.7	Verification Coverage	63
6.5.8	Runtime Tuning	63
6.5.9	Fault-Tolerance Improvements	63
6.5.10	Overall Outcome	64
6.6	Conclusion	64

6.7	Sprint Retrospective	64
7	General Conclusion	65
7.1	Summary	66
7.2	Objective Review	66
7.3	Future Work	67



List of Figures

1.1	DataDoIt company logo	3
1.2	Scrum process used to organize the project.	10
2.1	Global use case diagram of the queue wait-time estimation system. . . .	20
2.2	Overall system architecture from video acquisition to alert delivery. . . .	24
3.1	Sprint 1 activity flow from frame ingestion to queue metrics output. . . .	30
3.2	Sprint 1 sequence diagram from video ingestion to queue metric publica- tion.	31
4.1	Sprint 2 sequence diagram for feed orchestration and live event distribution.	37
4.2	Sprint 2 backend class diagram showing the core entities used by feed orchestration and persistence.	38
5.1	Sprint 3 sequence diagram for dashboard updates and alert surfacing. . .	45
5.2	Manager sign-in screen used to access the dashboard in Sprint 3.	47
5.3	Manager account creation screen introduced as part of the Sprint 3 au- thentication flow.	48
5.4	Administrator manager-administration screen for creating and maintain- ing manager accounts.	50
5.5	Manager dashboard showing the live monitoring surface, feed summary cards, and recent activity panel.	51
5.6	Feed setup screen used to prepare a local video source before zone tracing and model selection.	52

5.7	Zone tracing screen used to define the queue polygon for the selected feed.	53
5.8	Model selection screen used to choose the YOLO size before launching the feed.	54
5.9	Review and launch panel used to confirm the staged feed before creating the batch.	55
6.1	Actual Sprint 4 n8n workflow automation diagram used to route queue alerts through cooldown checks, archival, and Telegram delivery.	60



List of Tables

1.1	Collaboration perimeter with DataDoIt	3
1.2	From operational pain points to system-level requirements	5
1.3	Positioning of the proposed solution against existing approaches	7
1.4	Objective-to-module traceability	8
1.5	Scrum module mapping used in this report	11
1.6	Sprint planning used in the report	12
2.1	Authentication and access-control requirements	16
2.2	Functional requirement traceability matrix	17
2.3	Non-functional requirement quality matrix	18
2.4	Primary global use cases	20
2.5	Hardware environment	21
2.6	Execution environment profile	22
2.7	Development tools used in the project	22
2.8	Technologies used in the project	23
3.1	Sprint 1 backlog	29
4.1	Sprint 2 backlog	36
5.1	Sprint 3 backlog	44
6.1	Sprint 4 backlog	60



General Introduction

Service organizations are under constant pressure to reduce waiting time and improve service quality. Supermarkets, banks, and service centers all face the same problem: queues change quickly, are difficult to monitor by hand, and react strongly to demand fluctuations.

In practice, customers often join a queue without knowing how long they will wait, while managers and operational staff rarely have reliable real-time indicators for adjusting resources quickly. Manual observation is subjective, and simple counting tools are not enough to estimate waiting time in a meaningful way.

This end-of-studies project addresses that gap through the design and development of a real-time queue monitoring system based on computer vision and queueing theory. The proposed system detects and tracks people in video streams, analyzes occupancy inside configurable queue zones, estimates arrival and service rates, computes waiting time, and exposes metrics that can be used operationally.

It combines classical queueing theory with computer-vision detection and tracking techniques [1], [2] to provide actionable waiting-time estimates.

This report is organized as follows. Chapter 1 presents the project context, host organization, problem statement, and objectives. Chapter 2 defines the requirements analysis and the main architecture choices. Chapters 3 to 6 follow the implementation sprint by sprint, from perception and queue analytics to frontend integration and operational automation. The general conclusion closes the report and presents future work directions.

Context of the Project

Table of Content

1.1	Introduction	2
1.2	Host Organization	2
1.2.1	DataDoIt	2
1.2.2	Sectors of Activity	3
1.3	Presentation of the Project	4
1.3.1	Project Context	4
1.3.2	Problem Statement	4
1.3.3	Study of Existing Approaches	5
1.3.4	Proposed Solution	7
1.3.5	Objectives of the Project	8
1.3.6	Project Scope and Boundaries	9
1.3.7	Target Audience	9
1.4	Project Management Methodology	9
1.4.1	Choice of Methodology: Agile	9
1.4.2	SCRUM	10
1.4.3	Sprint Planning	11
1.5	Conclusion	13

1.1 Introduction

This chapter situates the Queue Wait-Time Estimation System in its operational and organizational context. It explains why queue monitoring remains important for service organizations and why a complete technical solution is needed instead of isolated counting tools.

It also sets the baseline used throughout the report: the collaboration with DataDoIt, the problem statement, the comparison with existing approaches, the solution scope, and the Scrum organization of the work. These points matter because the later technical chapters rely on them.

1.2 Host Organization

1.2.1 DataDoIt

This end-of-studies project was carried out with DataDoIt, a data-driven sustainability consulting firm headquartered at Elghazal Technopark in Tunis, Tunisia. DataDoIt works on projects that connect data science and sustainability for governments, municipalities, and enterprises across the EMEA region. Its services cover decarbonization strategy, ESG reporting, smart resource analytics, and circular economy implementation. The company uses the motto Measure It, Optimize It, Do It to describe a working method based on analysis, planning, and deployment. At the time of the project, it had delivered more than 15 active projects across 6 sectors, with a reported average ROI of 2.3x and a 94% client retention rate.

The collaboration provided a concrete setting for work centered on data collection, real-time monitoring, and automated analytics, and these concerns directly shaped the design and scope of the project. Figure 1.1 presents the DataDoIt company logo.



Figure 1.1: DataDoIt company logo

Table 1.1 summarizes the collaboration perimeter with DataDoIt.

Dimension	Scope in this project
Business context	Queue management in service scenarios where waiting time and staffing decisions are operationally sensitive.
Engineering constraints	Multi-source video ingestion, variable visual conditions, low-latency monitoring, and integration with automation channels.
Expected outputs	Actionable queue indicators, wait-time estimation, and alert-ready payloads for external workflows.
Validation mode	Iterative sprint reviews using implementation evidence from backend modules, API contracts, frontend behavior, and automation tests.

Table 1.1: Collaboration perimeter with DataDoIt

1.2.2 Sectors of Activity

DataDoIt delivers sustainability solutions across six mission-critical sectors where data-driven environmental performance has direct operational and financial implications:

- government and public sector institutions seeking to translate climate policy into measurable action,
- smart cities and municipalities requiring integrated urban resilience and climate adaptation systems,
- industrial and manufacturing operations targeting Scope 1-3 carbon footprint reduction and supply chain efficiency,
- real estate and infrastructure organizations pursuing green certification and climate-resilient asset development,

- financial services firms navigating ESG compliance and green finance opportunities,
- energy and utilities providers accelerating the transition toward renewable integration and grid modernization.

1.3 Presentation of the Project

1.3.1 Project Context

Manual queue supervision is often subjective, hard to scale, and dependent on continuous attention. In many settings, teams react only after congestion appears instead of anticipating it with measurable indicators.

This project addresses that gap by turning camera streams into decision-oriented telemetry. The target output is not limited to occupancy counts, but includes people in zone, arrival rate, service rate, expected wait time, queue stability state, and alert states that can be consumed by monitoring and automation layers.

1.3.2 Problem Statement

Current operational practices and basic counting tools still have recurring limitations:

- they provide limited anticipation of congestion,
- they rarely estimate waiting time explicitly,
- they provide weak visibility into queue stability during sudden demand spikes,
- they are often disconnected from automated response workflows.

As a result, interventions such as staff reallocation, lane opening, or escalation can be delayed or triggered on incomplete signals. The problem is therefore both analytical and integration-related: it is not enough to produce robust indicators; they also need to be

delivered through actionable channels. Table 1.2 maps these pain points to system-level requirements.

Observed pain point	Operational impact	Required system capability
Manual visual estimation is inconsistent across shifts.	Queue state interpretation varies between staff members, reducing decision reliability.	Standardized per-feed metrics with explicit formulas and stable update cadence.
Occupancy-only monitoring is descriptive but not predictive.	Teams react late because no forward-looking wait-time signal is available.	Continuous estimation of λ , μ , and expected wait time W .
Stability state is not explicit during bursty periods.	Teams may overreact or underreact when service and arrivals fluctuate quickly.	Stable/unstable queue-state classification with fallback wait-time logic.
Alerting tools are not integrated with runtime analytics.	Escalation depends on manual relay, increasing response latency.	Automated webhook dispatch and workflow-based notification routing.

Table 1.2: From operational pain points to system-level requirements

1.3.3 Study of Existing Approaches

Several computer-vision frameworks provide the basic building blocks for person detection and tracking. In parallel, queueing theory gives a formal way to describe service-system behavior and relate arrival rate, service rate, and expected waiting time [1].

In practice, existing commercial and open-source tools cover only parts of the queue-management problem. They fall into three broad families: queue management platforms, sensor-based analytics, and vision-based monitoring frameworks.

1.3.3.1 Queue Management Platforms

Platforms such as Waitwhile [3] and Qminder [4] focus on active queue management. They let customers join virtual queues through kiosks, QR codes, or web links, and they notify staff through SMS or dashboards. Their strength is customer-facing experience: reduced perceived waiting, appointment scheduling, and service routing. However, they require explicit customer interaction (check-in) and do not observe the physical queue directly. They cannot estimate wait time from the scene itself, and they do not detect congestion unless a customer has already registered.

1.3.3.2 Sensor-Based Analytics

RetailNext [5] and Xovis [6] represent the sensor-based analytics family. RetailNext uses ceiling-mounted AI sensors to track foot traffic, measure conversion rates, and optimize staffing in retail environments. Xovis uses 3D optical sensors for precise people flow and queue length measurement, particularly in airports and large retail spaces. Both provide high-accuracy occupancy and dwell-time data, but they require proprietary hardware installations and are primarily designed for large-scale enterprise deployments. They observe the queue passively but do not typically integrate with external automation workflows or provide open APIs for custom alert routing.

1.3.3.3 Vision-Based Monitoring Frameworks

Open-source tools built on YOLO detection and tracking, such as those provided by Ultralytics [7] and Roboflow [8], offer the computer-vision building blocks needed for queue monitoring: person detection, tracking, and zone-based counting. These frameworks are flexible and can be adapted to specific use cases. However, they typically stop at detection and counting. They do not include queue-theoretic estimation of wait time, stability classification, or integration with notification workflows. The gap between raw detection output and actionable operational metrics is left to the developer.

1.3.3.4 Positioning of the Proposed Solution

Table 1.3 positions the proposed solution against these existing approach families.

Approach family	Strengths	Typical limitation
Queue management platforms (Waitwhile, Qminder)	Customer-facing experience, virtual queues, appointment scheduling.	Require explicit check-in; do not observe physical queues or estimate wait time from the scene.
Sensor-based analytics (RetailNext, Xovis)	High-accuracy occupancy and flow data from dedicated 3D sensors.	Proprietary hardware; limited custom automation; enterprise-scale pricing.
Vision-based detection frameworks (Ultralytics, Roboflow)	Flexible open-source detection and tracking building blocks.	Stop at counting; no queue-theoretic estimation, stability classification, or alert integration.
Manual or occupancy-only monitoring	Easy to deploy and interpret.	Weak predictive value and high subjectivity.
Proposed integrated pipeline	Links perception, queue analytics, stability handling, web monitoring, and automation in one modular architecture.	Requires careful integration and validation across multiple technical domains.

Table 1.3: Positioning of the proposed solution against existing approaches

The proposed solution fills the gap between these families by combining open-source vision-based detection with queue-theoretic analytics, a web monitoring dashboard, and automated alert routing, all in one modular pipeline that does not require proprietary hardware or explicit customer interaction.

1.3.4 Proposed Solution

The system combines perception, analytics, and integration in one modular pipeline:

1. person detection and multi-object tracking from camera feeds,
2. polygon-based queue-zone filtering,
3. estimation of λ , μ , and expected wait time,
4. stable/unstable queue-state interpretation for safe operational decisions,
5. real-time web supervision through tiered live transport (WebRTC primary, MJPEG fallback),

6. automation through n8n webhooks and Telegram alerts.

This architecture converts visual observations into operational actions while keeping each layer independently testable. Detection and tracking modules can evolve without rewriting queue formulas, and the web and automation channels can evolve without changing the core metric contract.

1.3.5 Objectives of the Project

The project objectives are to:

1. detect and track people in real time,
2. count people in configurable queue zones,
3. estimate arrival and service rates,
4. compute expected wait time,
5. classify queue stability for operational reliability,
6. expose metrics through web interfaces and automation workflows.

To keep the implementation traceable, the objectives are mapped to delivery modules and observable outputs. Table 1.4 details the objective-to-module traceability.

Objective ID	Objective	Primary module	Observable output
OBJ-01	Real-time detection and tracking	Module A	Stable in-frame detections with persistent tracker IDs.
OBJ-02	Queue-zone counting	Module A	Per-frame in-zone people count and zone-aware overlays.
OBJ-03	Queue metric estimation (λ, μ, W)	Module A	Continuous metric payloads from queue analysis pipeline.
OBJ-04	Queue stability handling	Module A	Stable/unstable state classification and robust fallback wait-time behavior.
OBJ-05	Web monitoring	Module B	Live dashboard with feed controls, KPIs, and stream views.
OBJ-06	Automation integration	Module C	Webhook-based alert routing through n8n and Telegram.

Table 1.4: Objective-to-module traceability

1.3.6 Project Scope and Boundaries

The implementation scope covers backend perception and analytics modules, API and runtime integration, web monitoring, and external workflow automation. The report focuses on this web-first operational architecture.

All implementation details and validation narratives are presented through the backend, web, and automation tracks to keep the scope consistent.

1.3.7 Target Audience

The system is intended for three stakeholder groups with complementary goals:

- **Administrators:** manage manager accounts and access governance.
- **Managers:** configure operational settings, monitor live queue status, react to alerts, and use indicators for staffing and service-quality planning.
- **End users:** benefit indirectly from reduced waiting times and smoother service flow.

This separation is important because each group consumes different levels of abstraction, from per-feed live events to aggregated performance trends.

1.4 Project Management Methodology

1.4.1 Choice of Methodology: Agile

An Agile approach was selected to deliver functional increments quickly while reducing integration risks early. The project combines multiple technical domains, including computer vision, queue analytics, backend APIs, frontend workflows, and automation, and each one has uncertainties that are difficult to resolve in a single delivery cycle.

Iterative delivery made it possible to validate contracts progressively: first perception

outputs, then analytical reliability, then API/runtime integration, then web usability, and finally workflow automation and hardening.

1.4.2 SCRUM

Development followed sprint-based Scrum planning with iterative validation. For report readability and implementation traceability, the work is presented as four consolidated sprint chapters that are grouped into three logical modules.

The Scrum methodology is illustrated in Figure 1.2.

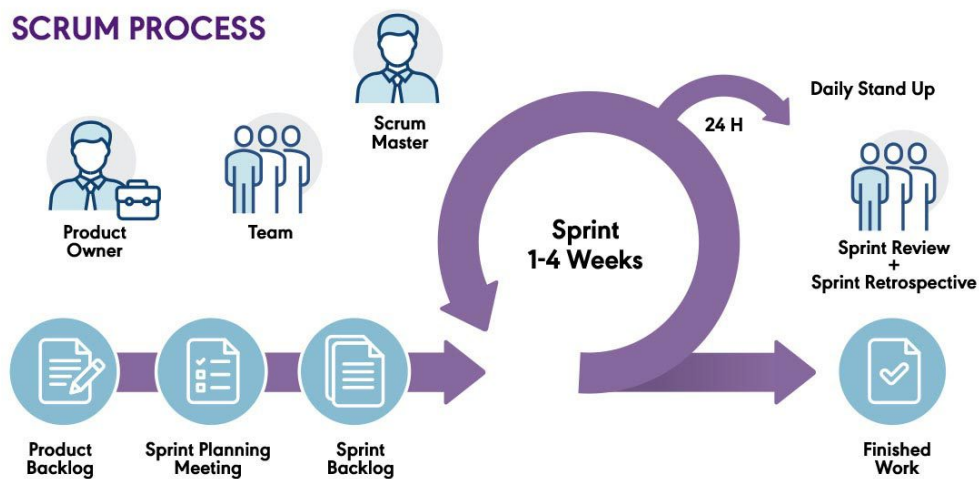


Figure 1.2: Scrum process used to organize the project.

1.4.2.1 Module A: Backend Core (Sprints 1–2)

This module established the vision, analytics, and backend service backbone:

- Sprint 1: detection, tracking, zone filtering, queue metrics, and stability handling,
- Sprint 2: camera connectivity, backend API exposure, and live transport.

1.4.2.2 Module B: Web Experience (Sprint 3)

This module focused on the web supervision experience:

- Sprint 3: web design system, frontend scaffolding, live backend integration, and dashboard monitoring.

1.4.2.3 Module C: Automation and Finalization (Sprint 4)

This module covered automation, evaluation, and project closure:

- Sprint 4: n8n automation, Telegram alerting, evaluation, optimization, and project consolidation.

Table 1.5 summarizes the Scrum module mapping used in this report.

Module	Sprints	Main focus	Validation perspective
Module A: Backend Core	1 to 2	Perception pipeline, queue analytics, camera connectivity, API services, live transport	Perception correctness, metric contracts, and API reliability.
Module B: Web Experience	3	Frontend design system, dashboard scaffolding, live integration, monitoring workflows	Usability, live-rendering stability, and role-based navigation.
Module C: Automation and Finalization	4	n8n alerting, Telegram notifications, evaluation, optimization, and fault tolerance	End-to-end operability and delivery readiness.

Table 1.5: Scrum module mapping used in this report

1.4.3 Sprint Planning

The project followed a sprint-based Scrum structure to keep the work incremental and traceable from requirements to implementation. Each sprint focused on one coherent part of the system, and the overall plan was adjusted as the internship progressed.

Table 1.6 details the sprint planning used in the report.

The detailed user stories for each sprint are presented in the backlog tables in Chapters 3 to 6.

Sprint	Objectives	Start Date	End Date	User Story IDs
Sprint 1	• Perception and queue analytics • Detection, tracking, and zone filtering • Wait-time estimation and stability handling	16/02/2026	14/03/2026	US1.1–US1.9
Sprint 2	• Camera connectivity and discovery • Backend API exposure • Live transport and feed management	15/03/2026	11/04/2026	US2.1–US2.4
Sprint 3	• Web design system and frontend scaffolding • Live backend integration • Dashboard monitoring and role-based navigation	12/04/2026	09/05/2026	US3.1–US3.9
Sprint 4	• n8n automation and Telegram alerting • Testing and verification • Runtime tuning and fault tolerance	10/05/2026	06/06/2026	US4.1–US4.7

Table 1.6: Sprint planning used in the report

1.5 Conclusion

This chapter set the project context, formalized the addressed operational problem, and justified the selected delivery strategy. Chapter 2 builds on this baseline by defining the requirements, architecture constraints, and analysis artifacts used to guide implementation decisions.

Table of Content

2.1	Introduction	15
2.2	Requirements Specification	15
2.2.1	Identification of Actors	15
2.2.2	Authentication and Access Governance	16
2.2.3	Functional Requirements	16
2.2.4	Non-functional Requirements	17
2.3	Product Backlog	18
2.4	Global Use Case Diagram	19
2.5	Sprint Scope Overview	21
2.6	Project Environment	21
2.6.1	Hardware and Software Environment	21
2.6.2	Development Tools	21
2.6.3	Technologies Used	23
2.7	Software Architecture	23
2.7.1	Overall Architecture	23
2.7.2	Backend Architecture	24
2.7.3	Frontend Architecture	25
2.8	Conclusion	25

2.1 Introduction

This chapter turns the project objective into the requirements that guided the implementation. It identifies the actors, lists the functions and quality constraints they need, and shows how those requirements are spread across the implementation plan.

The requirements are tied to the queue-monitoring pipeline used throughout the project: video ingestion, person detection and tracking, queue-metric estimation, dashboard delivery, authentication, and alert automation. This connection matters because the later chapters follow the same path from requirement to implementation, including the access layer used for manager onboarding and administrator governance.

2.2 Requirements Specification

2.2.1 Identification of Actors

The system has two actors whose responsibilities reflect the two main operating levels of the platform: governance and supervision. Both roles use the same application, but they do so with different permissions and goals.

The actors of the system are:

- **Administrator:** manages manager accounts (create, update, delete) and controls access governance.
- **Manager:** configures sources and thresholds, monitors live queues, reacts to alerts, and reviews performance indicators.

This split makes the functional scope easier to explain and validate, because access-control flows and operational monitoring flows can be tested separately.

2.2.2 Authentication and Access Governance

The platform also includes a lightweight identity layer so that the dashboard is not open to anonymous use. Managers can register, sign in, refresh their session, and log out, while administrators manage manager accounts and review the resulting actions. The detailed backend and frontend implementation appears in the sprint chapters, but the requirements are captured here because access boundaries are part of the core system behavior. Table 2.1 lists the authentication and access-control requirements.

ID	Requirement	Priority	Verification
AR-01	Allow a manager to register, sign in, refresh, and log out through secure cookies.	High	Validate register, login, refresh, logout, and session checks.
AR-02	Allow the administrator to create, update, deactivate, and reset manager accounts.	High	Validate account-governance endpoints and audit events.
AR-03	Enforce password policy, lockout after repeated failures, and role-based protection on operational endpoints.	High	Validate failed-login counters, lockout behavior, and unauthorized access.
AR-04	Record authentication and account-governance actions for later traceability.	Medium	Validate audit-log entries during register and login flows.

Table 2.1: Authentication and access-control requirements

2.2.3 Functional Requirements

At a high level, the system captures a feed, counts people, estimates queue behavior, updates the dashboard, and sends alerts when needed. From that workflow, the system must provide the following capabilities:

- real-time person detection and tracking,
- configurable queue zone definition,
- estimation of arrival rate (λ), service rate (μ), and wait time (W),
- queue stability classification with robust wait-time fallback behavior,
- real-time web dashboard updates and alerts,

- IP camera integration through complementary ONVIF discovery and RTSP streaming,
- automated webhook and Telegram notifications,
- secure manager sign-up, sign-in, and role-based access control,
- historical data logging and analytics.

These requirements define the minimum end-to-end behavior expected from the platform. To keep the implementation trackable, we formalize them in a traceability matrix. Table 2.2 presents the functional requirement traceability matrix.

ID	Requirement	Priority	Verification
FR-01	Detect people in live or recorded streams in near real time.	High	Validate detections and overlays in the perception layer.
FR-02	Preserve identity continuity across frames for queue analysis.	High	Validate tracker ID continuity in runtime scenarios.
FR-03	Restrict counting to configurable polygon queue zones.	High	Validate polygon inclusion in the zone-management layer.
FR-04	Compute λ , μ , and expected wait time from event windows.	High	Validate queue formulas and stability handling in analytics.
FR-05	Classify stable/unstable queue states with fallback wait-time logic.	High	Validate stability branches and fallback behavior.
FR-06	Expose feed control and live updates through API and websocket channels.	High	Validate endpoint and runtime behavior in API tests.
FR-07	Dispatch alert payloads to automation workflows.	Medium	Validate webhook dispatch and n8n branching.
FR-08	Persist operational metrics and metadata for analysis and auditing.	Medium	Validate persistence behavior in runtime and database checks.
FR-09	Support manager registration, sign-in, refresh, and logout through secure cookies.	High	Validate register, login, refresh, logout, and session checks.
FR-10	Protect administrative and operational APIs with role-based access control and audit logging.	High	Validate unauthorized access paths and audit events.

Table 2.2: Functional requirement traceability matrix

2.2.4 Non-functional Requirements

In addition to functional behavior, the platform must satisfy quality constraints that keep it usable in a live monitoring environment. Because the project combines computer vision, analytics, a web dashboard, and automation workflows, performance and

reliability matter as much as correctness. For this project, the main quality constraints are:

- **Performance:** maintain real-time behavior and low alert latency.
- **Scalability:** support multiple feeds and independent feed lifecycle control.
- **Reliability:** tolerate transient source failures and integration interruptions.
- **Usability:** provide clear web workflows for monitoring and configuration.
- **Security:** protect integration channels and service endpoints.
- **Maintainability:** keep modular code organization and testable interfaces.

We treat these quality attributes as engineering contracts rather than only design intentions. Table 2.3 summarizes the non-functional requirement quality matrix.

ID	Quality	Target indicator	Validation
NFR-01	Performance	Keep the loop near real time and limit alert delay.	Runtime observation under representative feed scenarios.
NFR-02	Scalability	Support independent control for multiple feed contexts.	Feed registry behavior and state-transition checks.
NFR-03	Reliability	Recover gracefully from camera drops and webhook failures.	Reconnect paths and retry / fallback test scenarios.
NFR-04	Usability	Keep setup and monitoring clear for non-developer users.	Dashboard integration tests and workflow walkthroughs.
NFR-05	Security	Restrict integration access through secrets and credentials.	Webhook secret validation and credential checks.
NFR-06	Maintainability	Keep the architecture modular and testable.	Code review and test-suite coverage by module.

Table 2.3: Non-functional requirement quality matrix

2.3 Product Backlog

The product backlog was prioritized according to business value and technical dependency. To keep the implementation easy to follow, the items were grouped into three domains that mirror the system architecture: backend processing, web supervision, and automation.

- **Backend domain:** perception pipeline, queue analytics, API and stream interfaces.
- **Web domain:** dashboard UX, live monitoring integration, zone editing workflows.
- **Automation domain:** n8n workflow integration, alert routing, webhook reliability.

The first sprints focused on backend foundations, because the dashboard and automation layers depend on a stable metric contract. This order reduced integration risk and made later validation clearer.

2.4 Global Use Case Diagram

The global use case diagram gives a high-level view of how the administrator and manager interact with the platform during configuration, monitoring, and alert handling. Figure 2.1 shows the global use case diagram of the queue wait-time estimation system.

Global Use Case Diagram

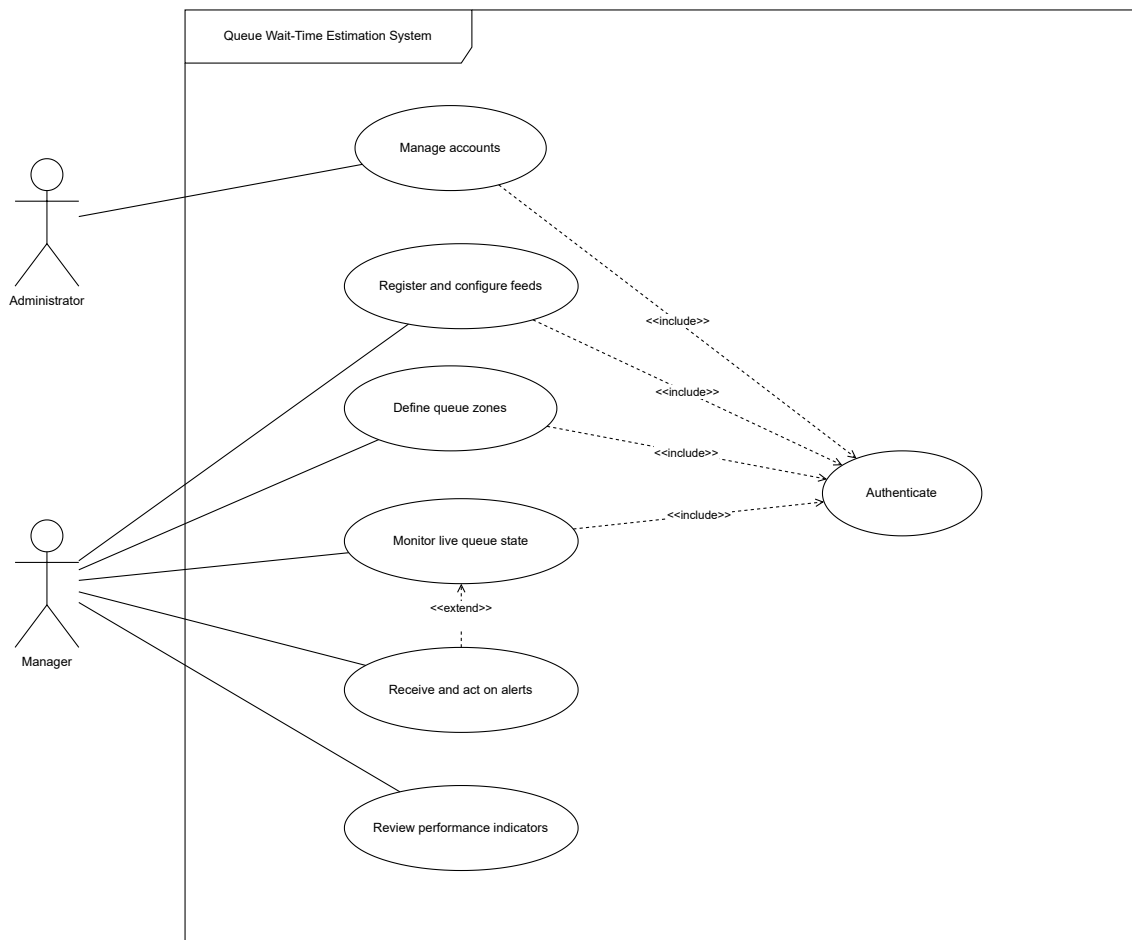


Figure 2.1: Global use case diagram of the queue wait-time estimation system.

At the system level, the use cases can be grouped into four interactions: configuration, live monitoring, analytics interpretation, and alert handling. Table 2.4 lists the use cases that should appear in the diagram and the main actor responsible for each one.

Use case ID	Use case summary	Primary actor
UC-01	Create, update, or remove manager accounts.	Administrator
UC-02	Register and configure a video feed source.	Manager
UC-03	Define or update queue zone geometry.	Manager
UC-04	Monitor live queue metrics and feed status.	Manager
UC-05	Receive and interpret warning or critical alerts.	Manager
UC-06	Review trends and performance indicators.	Manager

Table 2.4: Primary global use cases

2.5 Sprint Scope Overview

We organized the project into four consolidated sprints so that the pipeline could be delivered step by step and checked at each stage. The sprint chapters that follow describe those implementation phases in detail, from perception and queue analytics to frontend integration and automation.

2.6 Project Environment

2.6.1 Hardware and Software Environment

The hardware environment used during development is shown in Table 2.5. The software and runtime layers are summarized in Table 2.6. Table 2.6 summarizes the

Component	Type
Processor	Intel Core i5 12th Generation
Memory (RAM)	16 GB
Graphics card	NVIDIA GeForce RTX 4060
Storage	512 GB SSD

Table 2.5: Hardware environment

execution environment profile.

2.6.2 Development Tools

The project also relied on a small set of development tools for editing, versioning, diagramming, container support, and report generation. Keeping them separate from the technology stack makes it easier to distinguish what was used to build the project from what the project actually runs on. Table 2.7 lists the development tools used in the project.

Environment layer	Main tooling	Purpose
Backend runtime	Python ecosystem with vision and scientific libraries	Perception, queue analytics, and integration dispatch.
Frontend runtime	Node/Bun-based TypeScript toolchain with Vite	Dashboard rendering, interaction workflows, and live-state synchronization.
Automation runtime	Self-hosted n8n with webhook endpoints	Alert routing, cooldown logic, and external notifications.
Persistence/runtime support	SQLite database store (metrics, metadata, alerts)	Historical analysis and system state continuity.

Table 2.6: Execution environment profile

Tool	Role in the project	Typical use
VS Code	Main editor for the backend, frontend, and report sources.	Day-to-day implementation, review, and navigation.
Git / GitHub	Version control and change history tracking.	Commit management, branching, and collaboration.
draw.io	Diagram authoring for architecture and UML models.	Use case, UML class, and system architecture diagrams.
Docker / Docker Compose	Local service orchestration and reproducible setup.	Supporting services such as MediaMTX and n8n.

Table 2.7: Development tools used in the project

2.6.3 Technologies Used

The technology stack follows the main stages of the system and is summarized in Table 2.8.

Layer	Main technologies	Role in the project
Vision pipeline	YOLO26, ByteTrack, and OpenCV [2], [9], [10]	Detects people, tracks them across frames, and prepares the visual input for queue analysis.
Queue analytics	NumPy, Pandas, SciPy, and M/M/1 modeling [1]	Computes arrival rate, service rate, and wait time.
Backend API	FastAPI and Pydantic v2	Exposes typed endpoints, validates payloads, and coordinates the backend services.
Live transport	WebRTC, MediaMTX WHEP gateway, and MJPEG fallback [11], [12]	Delivers low-latency video streaming while keeping a simpler HTTP fallback available.
Frontend	React, TypeScript, Vite, and Tailwind CSS	Implements the dashboard, user interactions, and responsive presentation layer.
Automation	n8n workflows and Telegram notifications	Routes alerts and sends operational messages to users.
Persistence	SQLite	Stores metrics, metadata, and alert history for later analysis.

Table 2.8: Technologies used in the project

This selection keeps the implementation modular from perception through automation.

2.7 Software Architecture

2.7.1 Overall Architecture

We describe the architecture as four stages that match the flow of data through the system. Figure 2.2 illustrates the overall system architecture from video acquisition to alert delivery.

Chapter 2 System Architecture

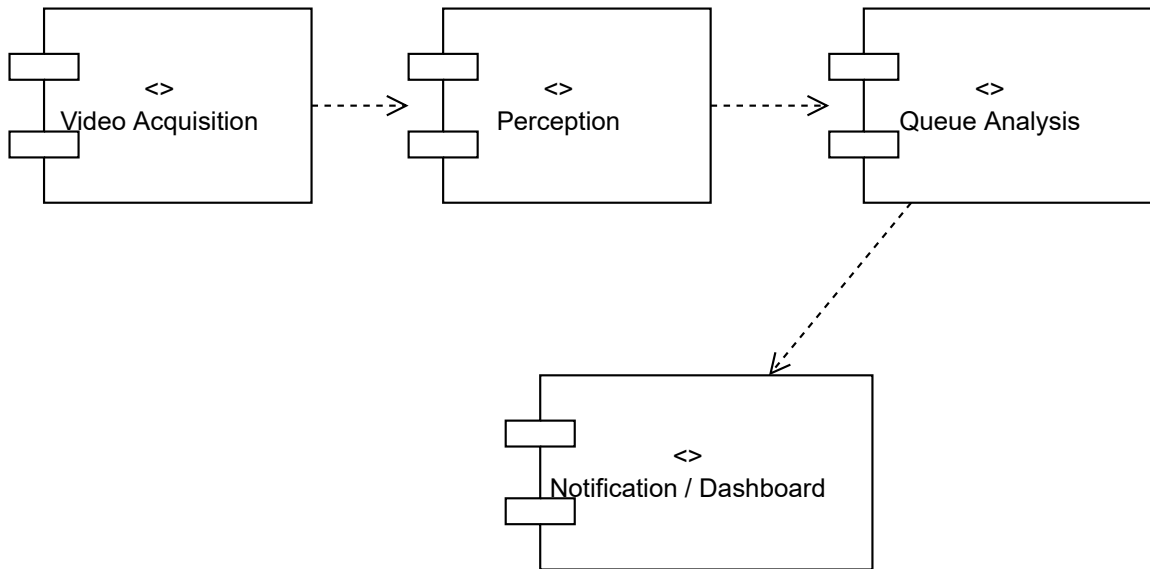


Figure 2.2: Overall system architecture from video acquisition to alert delivery.

1. **Video acquisition:** local files, RTSP streams, and ONVIF discovery.
2. **Perception:** detection, tracking, and queue-zone filtering.
3. **Analysis:** queue metrics estimation and stability handling.
4. **Output and integration:** web dashboard, webhook dispatch, and alert automation.

This separation keeps perception, analysis, the user interface, and automation independently testable. For example, queue formulas depend on perception events, but they do not depend on the frontend or notification delivery.

2.7.2 Backend Architecture

The backend exposes REST and websocket interfaces for feed lifecycle control and live metrics. It includes modules for detection orchestration, tracking, queue analysis,

threshold evaluation, webhook delivery, and camera-source management.

Feed-level orchestration lets each camera be started, stopped, or refreshed independently. If one feed fails, the others keep running, and the dashboard still receives a predictable event stream.

We treat persistence and integration as core responsibilities rather than add-ons: a durable backend state store preserves operational context, while webhook delivery turns queue events into external notifications.

2.7.3 Frontend Architecture

The frontend is built as a component-based interface with typed API integration, feed-oriented dashboard composition, and websocket-driven synchronization. Data access is isolated in hooks and client utilities, while pages and reusable components focus on presentation and interaction.

That separation keeps API changes localized to the integration layer and lets the visual components stay stable even when the backend contract changes. It also makes the dashboard easier to extend when new feed controls or indicators are added.

2.8 Conclusion

This chapter establishes the functional baseline, the quality constraints, and the architecture choices that guide the rest of the report. The next chapters turn these requirements into perception, connectivity, automation, and evaluation work.

Sprint 1: Perception and Queue Analytics

Table of Content

3.1	Introduction	27
3.2	Sprint Objectives	27
3.3	Sprint Backlog	28
3.4	Sprint Analysis	29
3.4.1	Runtime Scenarios	29
3.5	Design	30
3.5.1	Activity Diagram	30
3.5.2	Sequence Diagram	31
3.5.3	Operational Scenarios	31
3.6	Implementation	32
3.6.1	Detection Layer	32
3.6.2	Tracking Layer	32
3.6.3	Zone-Aware Filtering	32
3.6.4	Main Loop Integration	32
3.6.5	Queue Event Windows and Rate Estimation	32
3.6.6	Wait-Time Logic and Stability Handling	33
3.6.7	Runtime Metric Contract	33
3.7	Conclusion	34
3.8	Sprint Retrospective	34

3.1 Introduction

Sprint 1 built the full path from raw video to actionable queue indicators. It started by detecting people, tracking them across frames, and filtering observations to the queue zone. Once that perception baseline was reliable, the sprint moved to queue interpretation: estimating how fast people arrive, how quickly service handles them, and how long a new arrival should expect to wait.

This combination was deliberate. Perception without analytics produces only a count; analytics without stable perception produces unreliable rates. By delivering both in one sprint, the project established a trustworthy metric pipeline that every later sprint could consume directly.

3.2 Sprint Objectives

This sprint was focused on two complementary groups of outcomes:

Perception objectives:

- detect people reliably in live and recorded video,
- preserve identity continuity across frames,
- filter detections to the configured queue zone,
- deliver a stable perception baseline for analytics.

Analytics objectives:

- estimate arrival and service rates from tracked events,
- compute wait time in a way that remains meaningful during changing conditions,
- protect the runtime from empty or unstable windows,
- publish a metric contract that later layers can consume directly.

These objectives define the core of the system. If the perception layer is noisy or the analytics layer is fragile, every later metric and operational decision becomes harder to trust.

3.3 Sprint Backlog

The Sprint 1 backlog covers both perception and queue analysis tasks, organized so each user story traces from a stakeholder need to an implementation task. Table 3.1 presents the Sprint 1 backlog.

User StoryID	UserStory	TaskID	Task
US1.1	As a monitoring operator, I want to detect people in the video stream so that queue observations start from a reliable signal.	T1.1	Configure the detector to identify people in live and recorded video.
US1.2	As a monitoring operator, I want to keep the same person identifiable across frames so that counts remain consistent.	T1.2	Integrate tracking to preserve identity across consecutive frames.
US1.3	As a manager, I want to restrict counting to queue zones so that irrelevant movement does not affect the queue state.	T1.3	Define queue zones as polygons and filter detections by zone.
US1.4	As a developer, I want capture, detection, tracking, and display to work together so that the perception loop stays continuous.	T1.4	Connect capture, detection, tracking, zone filtering, and display into one flow.
US1.5	As a developer, I want basic runtime settings so that the pipeline can be adapted without code changes.	T1.5	Expose runtime settings for frame handling and visualization.

User StoryID	UserStory	TaskID	Task
US1.6	As a manager, I want queue rates estimated from tracked people so that the dashboard can interpret flow intensity.	T1.6	Group tracked events into short windows and compute arrival and service rates.
US1.7	As a manager, I want wait time computed for stable and unstable cases so that the result remains useful in real conditions.	T1.7	Apply the stable wait-time formula and fallback behavior.
US1.8	As a developer, I want the calculations protected from empty windows so that division-by-zero and noise do not break the runtime.	T1.8	Add guards for empty windows, low counts, and bounded fallback output.
US1.9	As a downstream consumer, I want a clear metric output so that later layers can reuse the values without reprocessing them.	T1.9	Publish rate estimates, wait time, people-in-zone, and stability status.

Table 3.1: Sprint 1 backlog

3.4 Sprint Analysis

3.4.1 Runtime Scenarios

Sprint 1 is best understood as a runtime pipeline that converts each frame into queue metrics. The scenarios below describe the conditions that the activity and sequence diagrams have to cover.

Four key scenarios are addressed in Sprint 1:

1. **Person entry into queue zone:** when a person enters the queue area, the system treats that person as a possible arrival.
2. **Consistency across frames:** when the same person appears in consecutive frames, the system keeps the identification stable and avoids double counting.
3. **Zone filtering under mixed scenes:** people outside the queue area are ignored so the queue state

stays focused on the relevant part of the scene.

4. **Queue stability under varying load:** when arrivals stay below service, the usual wait-time estimate is valid; when arrivals exceed service, the system reports a stability flag and uses a fallback value.

3.5 Design

3.5.1 Activity Diagram

Figure 3.1 illustrates the Sprint 1 activity flow from frame ingestion to queue metrics output.

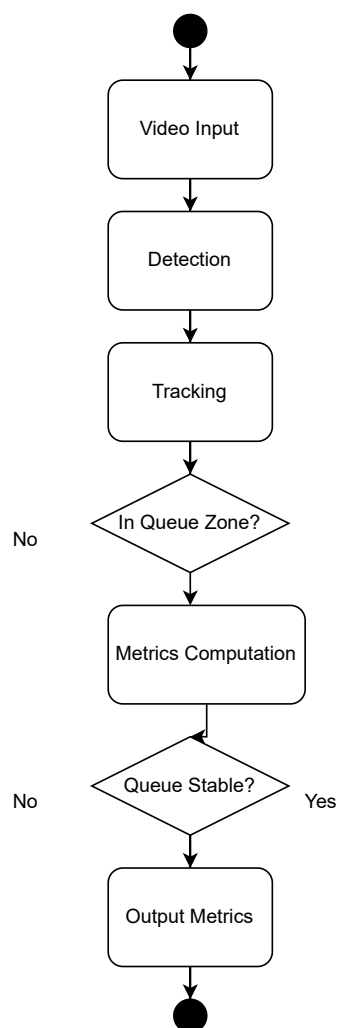


Figure 3.1: Sprint 1 activity flow from frame ingestion to queue metrics output.

The activity starts from a video source, whether it is a local file or a live camera stream. The system then detects people, keeps track of them, checks whether they are inside the queue area, and passes

the filtered result to the queue analysis stage, which computes arrival rate, service rate, wait time, and stability classification.

3.5.2 Sequence Diagram

Figure 3.2 shows the Sprint 1 sequence diagram from video ingestion to queue metric publication.

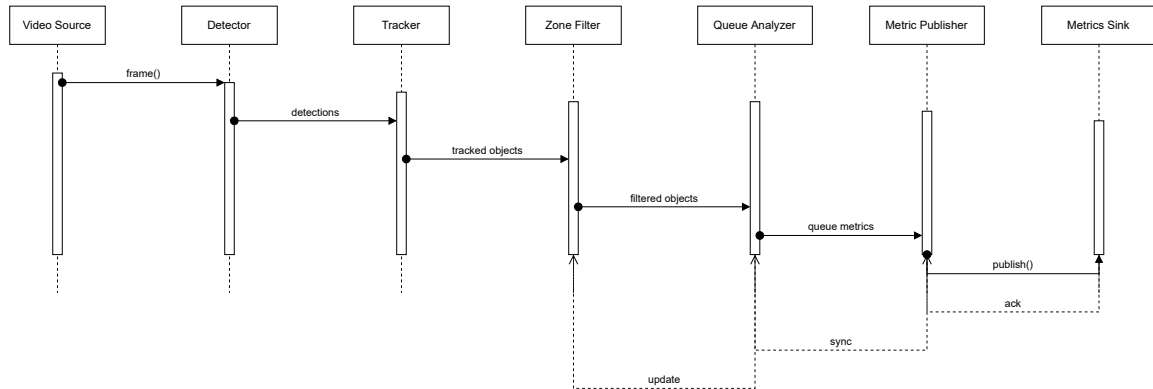


Figure 3.2: Sprint 1 sequence diagram from video ingestion to queue metric publication.

In practice, each frame moves through source handling, detection, tracking, zone checking, rate estimation, and metric publication. That ordered flow is what lets Sprint 1 deliver both a perception baseline and an analytical output in one pass.

3.5.3 Operational Scenarios

The queue analysis is easiest to understand through the situations it must handle every time a new metric window is evaluated:

1. **Stable queue:** arrivals stay below service, so the usual wait-time estimate is valid and can be forwarded to later layers.
2. **Growing backlog:** arrivals start to approach service, so the system keeps a stability flag and a fallback value instead of trusting a fragile instantaneous estimate.
3. **Empty or stalled queue:** no service events are available, so the output stays bounded and downstream logic can ignore the value rather than reacting to noise.

These scenarios explain why the sprint reports both point estimates and a stability signal instead of returning a raw number with no operational context.

3.6 Implementation

3.6.1 Detection Layer

The detection layer focuses on finding people in each frame while keeping the process adjustable for different machines. It is tuned to balance speed and accuracy, so the system stays responsive without losing the main detections that matter for the queue.

3.6.2 Tracking Layer

The tracking layer keeps the same person identifiable from one frame to the next. This is what makes arrival and departure counts more reliable than simply reading detections in a single frame [9].

3.6.3 Zone-Aware Filtering

Zone logic defines the queue area as a polygon on the frame and keeps only the people inside it. This avoids counting unrelated movement elsewhere in the scene and makes the measurement more relevant to the waiting line.

3.6.4 Main Loop Integration

The pipeline integration is handled in the runtime orchestration layer. The runtime supports:

- handling local files, webcams, and live camera streams,
- adjusting how often frames are processed so the system stays responsive,
- showing detections, IDs, the queue zone, and metrics on screen,
- running without a visible interface when needed for later integration.

3.6.5 Queue Event Windows and Rate Estimation

The queue-analysis step uses short time windows to count arrivals and departures. Let N_a and N_d be the number of events observed in windows T_a and T_d :

$$\lambda = \frac{N_a}{T_a}, \quad \mu = \frac{N_d}{T_d}$$

From these counts, the system estimates the arrival rate and the service rate. To avoid unstable values at the beginning of a run, the implementation waits until enough events have been observed before returning non-zero rates.

3.6.6 Wait-Time Logic and Stability Handling

The queue is considered stable when arrivals stay below service and service is actually observed:

$$\text{queue_stable} = (\lambda < \mu) \wedge (\mu > 0)$$

When that condition is met, the normal wait-time formula is used. Otherwise, the system falls back to a more conservative estimate based on the current occupancy:

- stable queue: $W = \frac{1}{\mu - \lambda}$,
- unstable but service observed: fallback $W = \frac{L}{\mu}$, where L is the number of people in the zone.

We use the M/M/1 sojourn-time expression because the project models each monitored lane as a single service channel with one effective queue, which matches the cashier-style setting studied in the report. This formula is also easy to interpret during a defense: it depends directly on the measured arrival and service rates, it gives a closed-form estimate without needing a large historical dataset, and it stays aligned with the real-time pipeline used by the tracker. Other models were less appropriate here: M/M/c would assume multiple parallel servers, which is not the default lane setup, and Little's Law is useful for validation but does not directly produce a forward wait-time estimate from the instantaneous rates alone. When no service events are observed, the implementation returns a zero fallback so that downstream components do not use an unreliable value.

3.6.7 Runtime Metric Contract

The final metric output includes:

- point estimates (λ, μ, W) ,
- people-in-zone occupancy,
- stability flag for safe downstream interpretation.

These values are then reused by logs, overlays, webhook payloads, and dashboard events [1], [10].

3.7 Conclusion

Sprint 1 delivered the full path from raw video to operational queue indicators. People can be detected, followed across frames, counted inside the queue area, and the resulting events are turned into arrival rates, service rates, wait-time estimates, and stability classifications. That gives the next sprint a trustworthy analytical contract to build on for camera integration and API services.

3.8 Sprint Retrospective

The main achievement of this sprint was connecting perception to interpretation in one delivery. The tracker and zone logic reduce false continuity problems, while the queue model provides both formulas and guardrails. A mathematically correct wait-time result is not enough if the input window is empty or the queue is unstable, so the implementation had to treat safety and interpretability as equal concerns alongside detection accuracy.

The value of this sprint is that it gives the rest of the system a trustworthy metric layer. Once the queue state is expressed through rates, stability, and fallback behavior, the dashboard and automation work can operate on structured signals instead of raw counts.

Table of Content

4.1	Introduction	36
4.2	Sprint Objectives	36
4.3	Sprint Backlog	36
4.4	Design	37
4.4.1	Sequence Diagrams	37
4.4.2	Backend Class Diagram	37
4.5	Implementation	38
4.5.1	Camera Source Integration	38
4.5.2	Authenticated API Access	39
4.5.3	Camera Discovery and Stream Setup	39
4.5.4	API Layer and Data Contract	39
4.5.5	Live Video Delivery	39
4.5.6	Runtime Control and Live Events	40
4.5.7	Backend Persistence Architecture	40
4.5.8	Persistence and Recovery	40
4.6	Conclusion	40
4.7	Sprint Retrospective	41

4.1 Introduction

Sprint 2 expanded the project from a single local pipeline into a platform that can work with several camera sources. At this stage, the backend became responsible for camera onboarding, feed control, and live delivery, so the rest of the application could communicate with it in a more structured way.

4.2 Sprint Objectives

This sprint had four main delivery goals:

- make external camera sources usable inside the project,
- expose feed management through a predictable API contract,
- keep live delivery available through a fallback-aware transport strategy,
- preserve feed state and recovery behavior across restarts.

These goals are important because this sprint turns the backend into a reusable service layer rather than a one-off processing script.

4.3 Sprint Backlog

The Sprint 2 backlog was organized as a table so the connectivity work can be traced from camera onboarding to API exposure. Table 4.1 presents the Sprint 2 backlog.

User StoryID	UserStory	TaskID	Task
US2.1	As a manager, I want IP camera streams connected securely so that live sources can be reused reliably.	T2.1	Add authentication, transport control, reconnection, and snapshots for RTSP feeds.
US2.2	As a manager, I want cameras discovered automatically so that stream addresses do not need to be hardcoded.	T2.2	Detect IP cameras and resolve their stream addresses.
US2.3	As an operator, I want feed management exposed through the API so that camera lifecycle actions are centralized.	T2.3	Expose feed control, metadata handling, and diagnostic operations.
US2.4	As a user, I want live delivery to stay available so that the dashboard can keep showing the active feed.	T2.4	Support live delivery through real-time events and a layered video transport strategy.

Table 4.1: Sprint 2 backlog

4.4 Design

4.4.1 Sequence Diagrams

Figure 4.1 illustrates the Sprint 2 sequence diagram for feed orchestration and live event distribution.

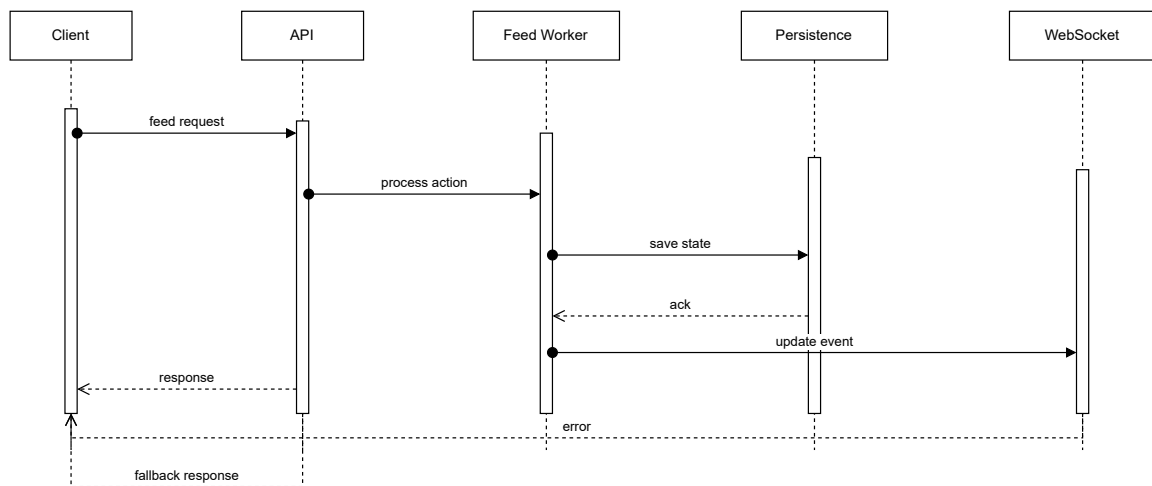


Figure 4.1: Sprint 2 sequence diagram for feed orchestration and live event distribution.

The running sequence is straightforward: a frontend action reaches the backend, the feed state is updated, the worker process changes accordingly, and the new status is sent back to the live interface.

4.4.2 Backend Class Diagram

Figure 4.2 shows the Sprint 2 backend class diagram used by feed orchestration and persistence.

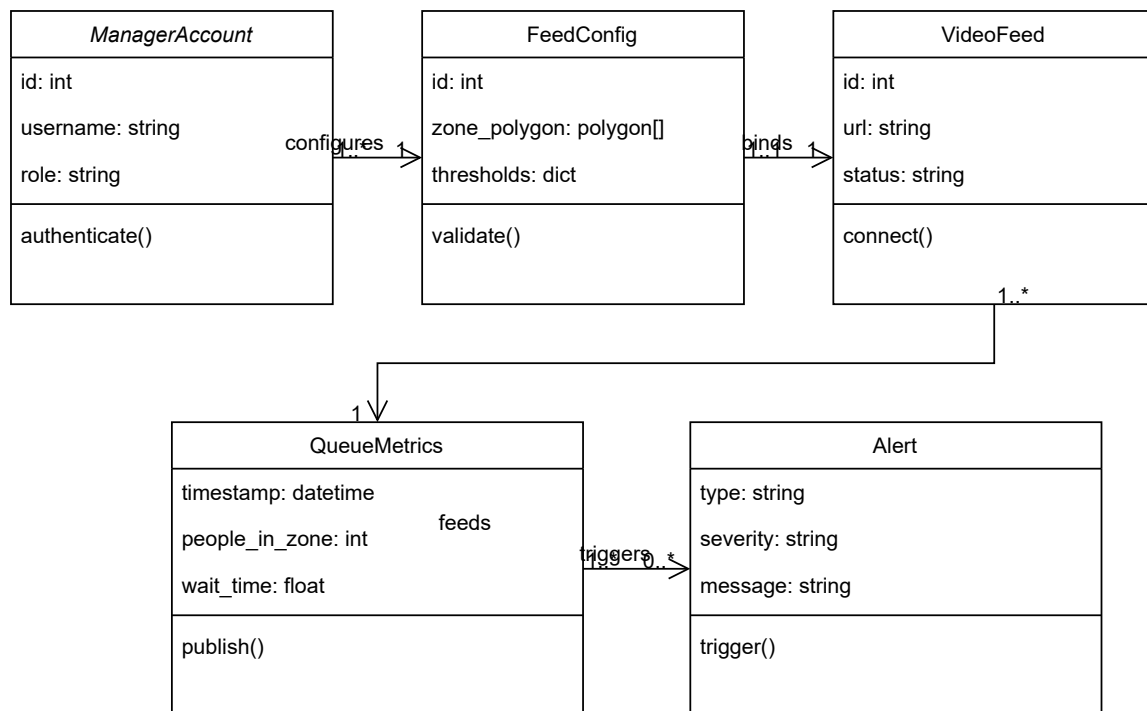


Figure 4.2: Sprint 2 backend class diagram showing the core entities used by feed orchestration and persistence.

This structural view complements the sequence diagram by showing how the backend keeps manager accounts, feed configurations, video feeds, queue metrics, and alerts in a cohesive domain model. It is especially relevant in Sprint 2 because the chapter introduces authenticated API access, SQLite persistence, and feed-state recovery.

4.5 Implementation

4.5.1 Camera Source Integration

This part of the sprint focuses on making external cameras usable inside the project. It supports:

- handling camera credentials without exposing them in logs,
- choosing between TCP and UDP transport depending on the situation,
- checking the connection and capturing snapshots,
- reconnecting automatically when the stream is interrupted.

The source-handling layer routes local files and webcams through one path and live camera streams through another, which keeps the source logic easier to maintain.

4.5.2 Authenticated API Access

Sprint 2 also introduced session-aware access control in the backend API. Managers can only interact with the feeds that belong to their account, while administrators keep the broader management scope needed for account governance. The same identity context is applied to feed listing, feed creation, feed lifecycle actions, stream snapshots, and websocket subscriptions so the runtime respects the ownership rules defined earlier in the report.

4.5.3 Camera Discovery and Stream Setup

The camera discovery step is used to find available cameras automatically and recover their stream information. This avoids hardcoding stream addresses and makes onboarding easier when new IP cameras are added.

In the practical setup, the workstation and the IP camera were connected to the same local network through a router, which made the device reachable for discovery and RTSP validation. The test source used during this stage was a Digital Watchdog IP camera, allowing the connectivity workflow to be validated against a real camera feed.

4.5.4 API Layer and Data Contract

The API layer acts as the bridge between the backend services and the rest of the project. It provides structured inputs and outputs so the frontend can interact with the backend in a predictable way. The main groups of operations are:

- feed management, including creation, listing, start, stop, restart, and deletion,
- source onboarding, including RTSP checks, snapshots, and camera discovery,
- metadata management for establishments and cashiers,
- integration and health checks,
- live transport support for real-time viewing.

4.5.5 Live Video Delivery

To reduce viewing latency while still keeping a fallback path, Sprint 2 introduces a transport system that chooses the most suitable stream mode for each feed [11], [13]. The backend tells the frontend

whether live viewing is ready, whether WebRTC is available, and which fallback should be used when the higher-quality path cannot start. When WebRTC is available, the backend forwards the browser offer to the media server and returns the response needed to start the stream [12]. If the feed does not exist, is not running, or the media service cannot answer, the backend reports the condition clearly so the frontend can switch to the fallback view instead of failing silently. The configuration stays flexible through environment settings such as the media URL, control URL, timeout, and feature flag.

4.5.6 Runtime Control and Live Events

Runtime control sits inside the API orchestration layer. It keeps track of feed state, synchronizes persistence, manages the worker lifecycle, and dispatches events to the rest of the system. The live channel publishes updates such as snapshots, feed status changes, metric updates, alert notifications, and system warnings.

For the video surface, the system keeps a shared fallback stream so that viewing does not break when the runtime mode changes.

4.5.7 Backend Persistence Architecture

The backend persistence architecture is implemented in SQLite, with a schema that preserves establishments, checkout counters, feed configurations, video sessions, alerts, users, refresh sessions, and auth audit records. This storage design supports both operational recovery and historical analysis, while the runtime layer uses it to restore stale sessions and preserve manager metadata across restarts.

4.5.8 Persistence and Recovery

The persistence layer stores feed configuration, metadata, video sessions, and archived alerts in SQLite. When the backend starts, it reloads saved feeds and safely resets any stale running sessions so the system begins from a consistent state.

4.6 Conclusion

By the end of Sprint 2, the backend could onboard cameras, manage feed state, and send live updates to the frontend without depending on hardcoded source details. That made the web UI work in Sprint 3 much easier to integrate.

4.7 Sprint Retrospective

This sprint clarified the role of the backend as the integration hub. It now handles camera identity, live transport, recovery, metadata, and state synchronization in one place, which reduces coupling in the rest of the system.

The main lesson is that connectivity is not only about reaching the camera. It is also about preserving a usable application state when streams fail, restart, or need to switch transport modes.

Table of Content

5.1	Introduction	43
5.2	Sprint Objectives	43
5.3	Sprint Backlog	43
5.4	Design	45
5.4.1	UI and UX Design	45
5.4.2	Sequence Diagrams	45
5.4.3	Design Rationale	46
5.5	Implementation	46
5.5.1	Project Setup and Layout Skeleton	46
5.5.2	Styling and Reusable Interface Rules	46
5.5.3	API Integration	46
5.5.4	Authentication and Role-Based Navigation	47
5.5.5	Dashboard State Orchestration	48
5.5.6	Live Update Reliability	49
5.5.7	Live Video Delivery	49
5.5.8	Dashboard UI Integration	50
5.5.9	Integration Testing	55
5.6	Conclusion	56
5.7	Sprint Retrospective	56

5.1 Introduction

Sprint 3 covered the frontend work from the first design decisions and page structure to the live backend integration that made the dashboard operational. The sprint started by defining the visual direction and navigation model, then connected the interface to backend services and runtime events so the dashboard could react to live data, transport changes, and feed lifecycle actions.

This two-phase approach was intentional. Settling the design system and page skeleton first meant that the live integration work could focus on behavior and reliability instead of redesigning the interface at the same time.

5.2 Sprint Objectives

This sprint was organized around six outcomes spanning design and live integration:

Design and scaffolding objectives:

- define a coherent visual language for the dashboard,
- prepare the main page layouts before live data integration,
- build reusable UI structures that can support later features.

Live integration objectives:

- make the dashboard consume the backend contract without ad hoc transformations,
- keep the interface responsive when live transport is available, delayed, or interrupted,
- validate that monitoring, feed management, and fallback behavior remain understandable for the operator.

5.3 Sprint Backlog

The Sprint 3 backlog was organized as a table so both the design foundation and the live monitoring tasks remain traceable. Table 5.1 presents the Sprint 3 backlog.

SPRINT 3: WEB DASHBOARD AND LIVE MONITORING

User StoryID	UserStory	TaskID	Task
US3.1	As a user, I want a consistent visual style so that the dashboard feels like one coherent product.	T3.1	Define colors, typography, spacing, and reusable component rules.
US3.2	As a manager, I want the main page layouts prepared so that onboarding, monitoring, editing, and analytics screens are available.	T3.2	Prepare the landing page, dashboard, setup wizard, zone editor, analytics, and settings layouts.
US3.3	As a developer, I want reusable frontend elements so that the interface can grow without duplication.	T3.3	Set up the frontend project with shared components and page scaffolding.
US3.4	As a user, I want the dashboard connected to backend responses so that the interface reflects live operational data.	T3.4	Implement the API integration layer and typed request handling.
US3.5	As a manager, I want to manage feeds from the interface so that feed lifecycle actions stay centralized.	T3.5	Connect login state, protected routes, and feed control actions to the dashboard.
US3.6	As a user, I want live updates without refreshing so that changes are visible immediately.	T3.6	Add websocket-driven live state updates and dashboard orchestration.
US3.7	As a user, I want a suitable video transport chosen automatically so that the feed remains usable when conditions change.	T3.7	Add transport selection, fallback streaming, and reconnection logic.
US3.8	As a user, I want the dashboard to stay usable during interruptions so that monitoring does not stop during transient failures.	T3.8	Add fallback polling and stale-data guards.
US3.9	As a developer, I want setup and onboarding flows validated so that the integration behaves correctly end to end.	T3.9	Validate dashboard setup, feed onboarding, and error behavior with integration tests.

Table 5.1: Sprint 3 backlog

5.4 Design

5.4.1 UI and UX Design

The project design specification introduces a “Terminal Minimal” style and a seven-page navigation model. The goal was to give the dashboard a simple visual identity that fits a monitoring application. The design language emphasizes:

- dark operational surfaces optimized for continuous monitoring,
- strong semantic colors for status and alert levels,
- card-based information density with consistent spacing and hierarchy,
- explicit workflow guidance in setup and zone-editing screens.

The designed page set includes the landing page, dashboard, setup wizard, zone editor, analytics, settings, and heatmap views. These screens helped define what the frontend had to support before live data arrived.

5.4.2 Sequence Diagrams

Figure 5.1 illustrates the Sprint 3 sequence diagram for dashboard updates and alert surfacing.

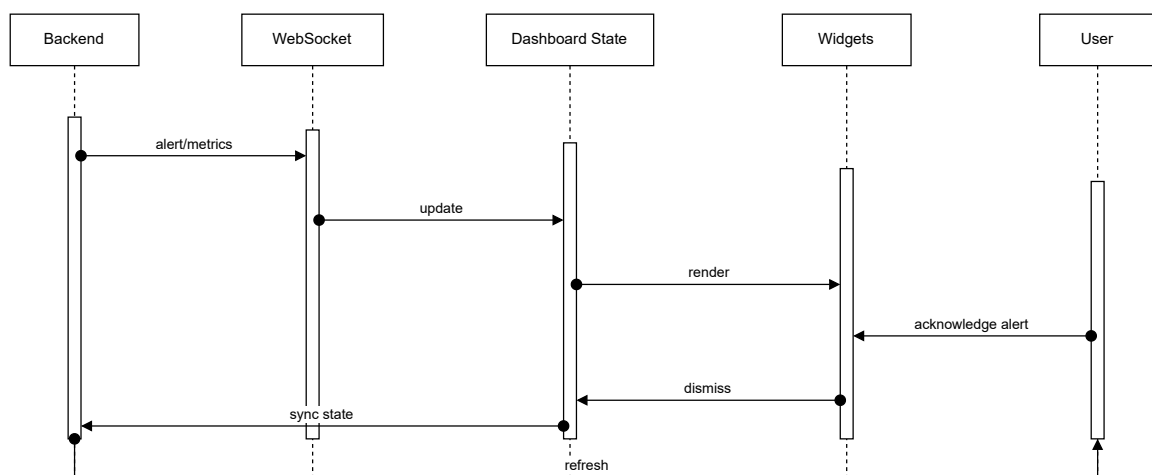


Figure 5.1: Sprint 3 sequence diagram for dashboard updates and alert surfacing.

Three main flows were implemented:

1. **Feed lifecycle:** a dashboard action updates the feed state and the new status is sent back to the interface.

2. **Live metrics:** metric updates are received from the backend and reflected immediately on the dashboard.
3. **Alert surfacing:** alerts reach the interface and appear in the activity and attention areas.

5.4.3 Design Rationale

The design phase was kept as a foundation step rather than a full UI release. The goal was to settle navigation, component boundaries, and interaction patterns before live data and transport logic arrived. A clean shell was a useful deliverable because the integration work depended on it.

5.5 Implementation

5.5.1 Project Setup and Layout Skeleton

The implementation started by turning the design specification into an application skeleton. The workspace setup established the application entry point, the navigation structure, and the shared layout that all pages reuse.

The order of the pages was not arbitrary. The layout had to support a workflow that begins with onboarding, continues through configuration, and ends with live supervision. That is why the page skeleton was treated as an operational path rather than a list of unrelated screens.

5.5.2 Styling and Reusable Interface Rules

The visual system was defined once and then applied consistently across the dashboard. Spacing, typography, surface treatment, and semantic status colors were treated as reusable rules rather than per-page decisions. This reduced design drift: when a status card, a table, or a setup control appears in several places, it behaves like the same product element instead of being restyled each time.

5.5.3 API Integration

The frontend relies on a dedicated API layer that keeps the communication with the backend structured. It defines the main data types used by the dashboard and centralizes request handling so errors can be managed consistently. This layer acts as the contract boundary: it prevents the UI from depending

directly on backend implementation details and makes it easier to reason about feed lists, dashboard summaries, and management actions as explicit typed responses.

5.5.4 Authentication and Role-Based Navigation

Sprint 3 connected the login and sign-up flows to the dashboard. The auth provider clears cached data when the session changes, protected routes redirect users according to their role, and administrators are taken to the manager administration page while managers are routed to the monitoring dashboard. This keeps the frontend aligned with the backend access rules and avoids exposing the wrong interface to the wrong user. Figure 5.2 shows the manager sign-in screen used to access the dashboard in Sprint 3.

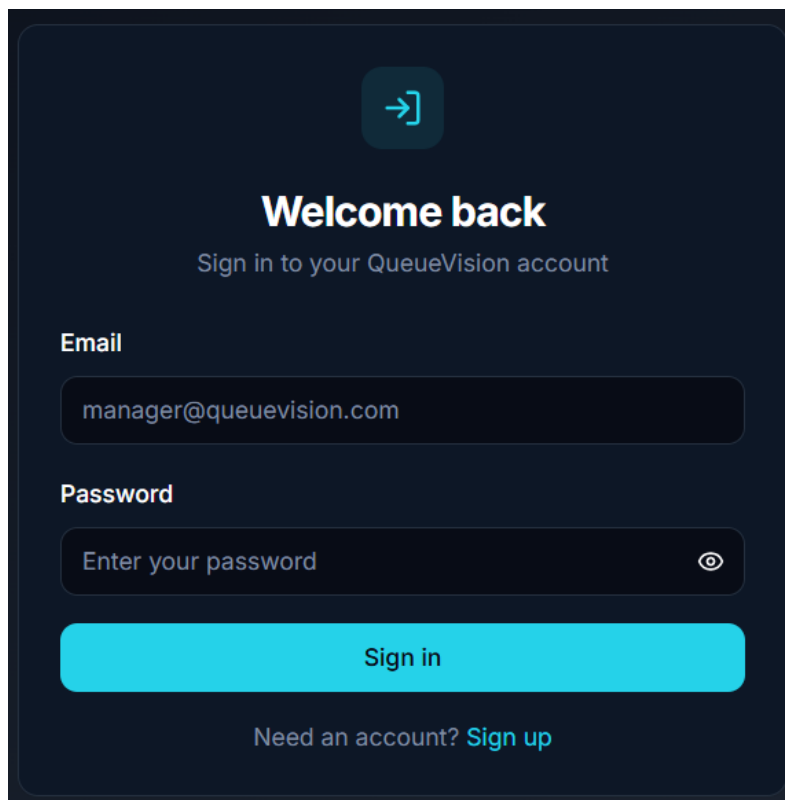
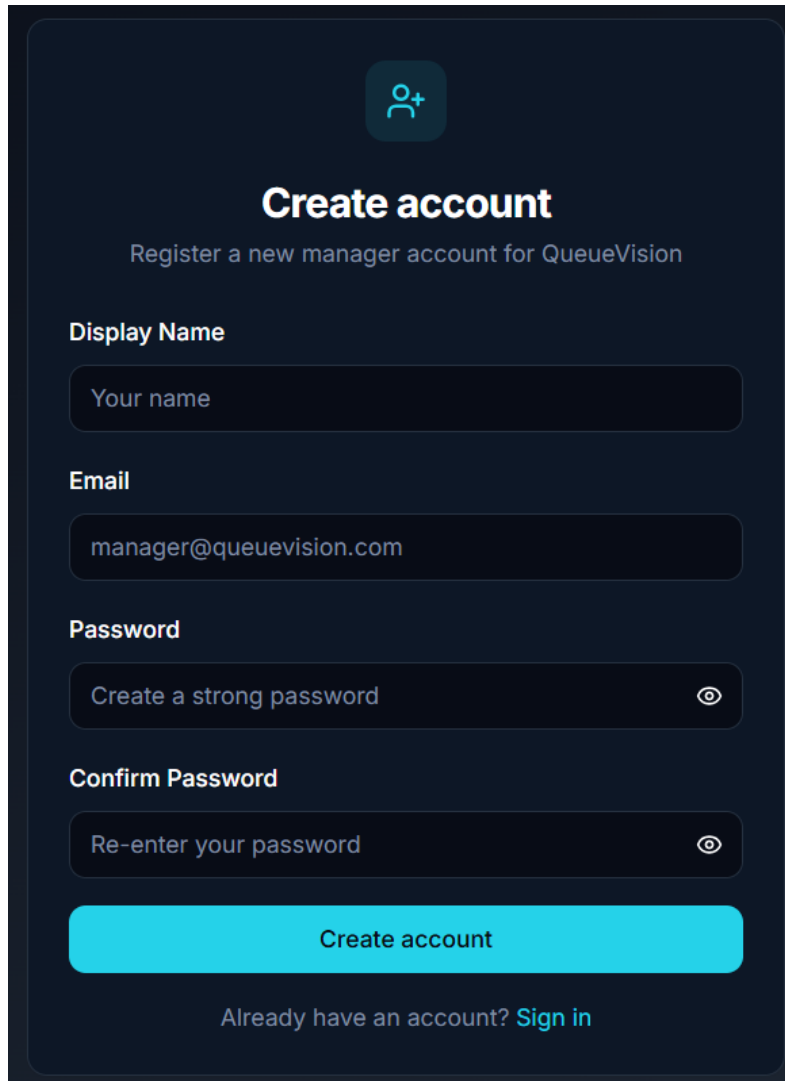


Figure 5.2: Manager sign-in screen used to access the dashboard in Sprint 3.

Figure 5.3 presents the manager account creation screen introduced as part of the Sprint 3 authentication flow.

The image shows a dark-themed web form for creating a manager account. At the top, there is a teal icon of a person with a plus sign. Below it, the title "Create account" is displayed in white, followed by the subtitle "Register a new manager account for QueueVision" in a smaller, lighter font. The form contains four input fields: "Display Name" with the placeholder "Your name", "Email" with the placeholder "manager@queuevision.com", "Password" with the placeholder "Create a strong password", and "Confirm Password" with the placeholder "Re-enter your password". Each password field has a small eye icon to toggle visibility. At the bottom of the form is a large teal button labeled "Create account". Below the button, there is a link that says "Already have an account? Sign in".



Create account

Register a new manager account for QueueVision

Display Name

Email

Password

Confirm Password

[Create account](#)

Already have an account? [Sign in](#)

Figure 5.3: Manager account creation screen introduced as part of the Sprint 3 authentication flow.

5.5.5 Dashboard State Orchestration

A dedicated dashboard state layer coordinates data fetching and updates. From the received feed data, the dashboard derives indicators such as online feeds, total people, average wait time, and attention counts, then exposes them to the widgets.

The state layer also helps absorb transient data gaps. When a websocket event arrives late, the dashboard can keep displaying the last valid snapshot instead of dropping the screen into an empty state. That design choice improves usability during unstable network periods and keeps the monitoring surface trustworthy.

5.5.6 Live Update Reliability

The live-update layer includes:

- reconnection delays,
- a heartbeat check,
- a guard against stale data,
- fallback polling when live updates fail,
- grouped updates to keep the interface responsive.

When the websocket connection drops, the dashboard falls back to periodic polling instead of freezing, so operators still have a usable view during transient issues.

This fallback behavior was treated as a first-class feature rather than an error path. In a queue-monitoring context, the operator needs continuity more than perfect transport purity, so the UI must remain informative even when the preferred live path is temporarily unavailable.

5.5.7 Live Video Delivery

Live feed cards use a transport decision layer to choose the most suitable viewing mode. For each running feed, the frontend checks whether live viewing is ready. If the higher-quality live stream is available, it is used first; otherwise the dashboard falls back to the simpler video stream.

The live-streaming flow is handled through the browser's live video connection support [11]:

- create a connection for receiving video,
- gather the network information needed for signaling,
- send the browser offer to the backend,
- apply the backend response and switch the feed card to live state.

Reconnection uses increasing delays between attempts. When the problem is not recoverable, the system stops retrying and switches to the fallback stream so the dashboard remains usable.

When server-side annotations are active, the dashboard hides local overlays to avoid drawing the same boxes twice on the same video.

5.5.8 Dashboard UI Integration

The dashboard page brings together live feed controls, setup dialogs, source onboarding, KPI visualizations, and charts. The emphasis was on making those elements work together as one operational interface.

The practical result is that a manager can move from onboarding to monitoring without leaving the dashboard context. That continuity mirrors the workflow described in the earlier requirements chapter. Figure 5.4 shows the administrator manager-administration screen for creating and maintaining manager accounts.

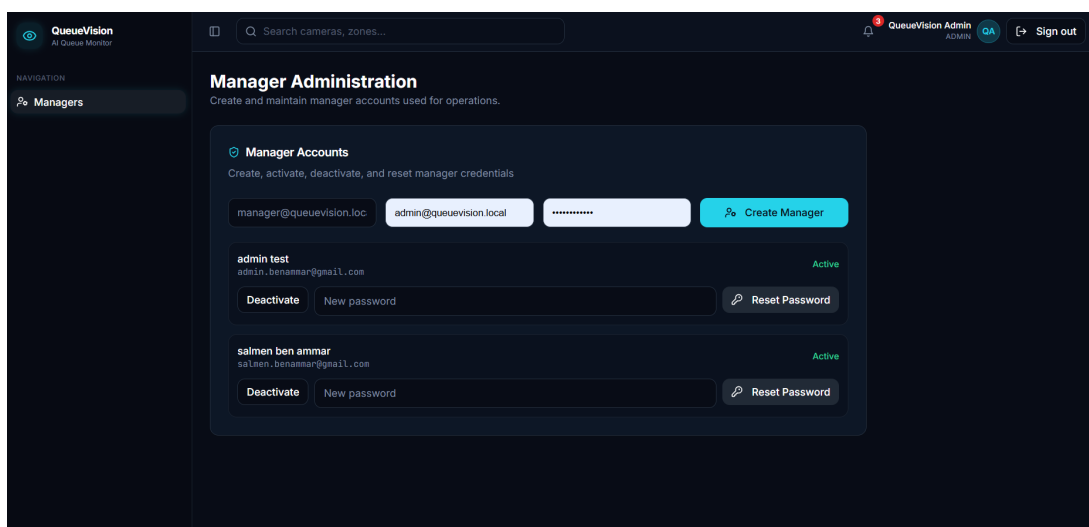


Figure 5.4: Administrator manager-administration screen for creating and maintaining manager accounts.

Figure 5.5 presents the manager dashboard showing the live monitoring surface, feed summary cards, and recent activity panel.

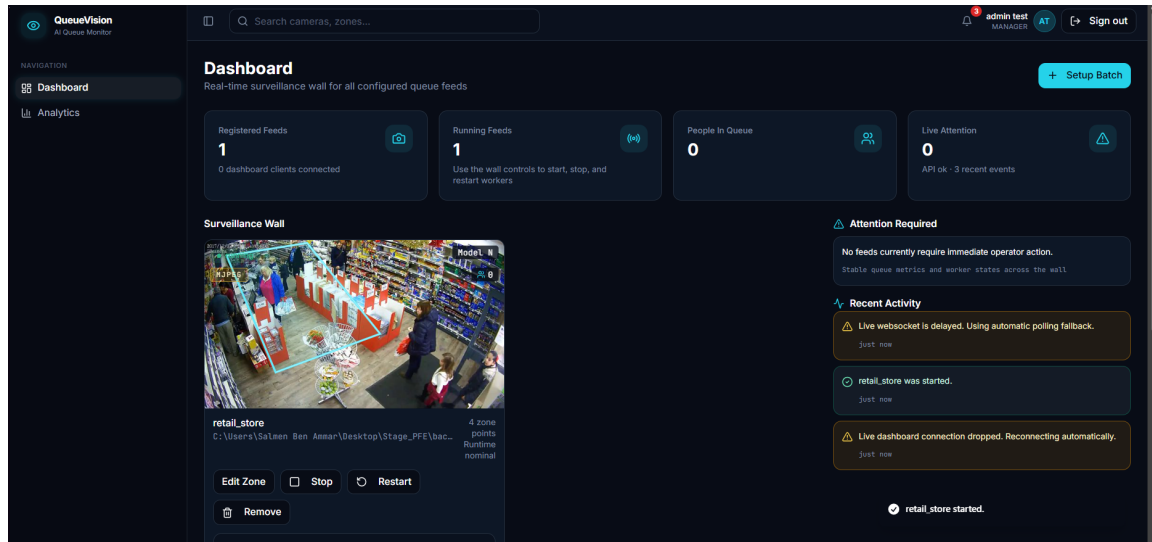


Figure 5.5: Manager dashboard showing the live monitoring surface, feed summary cards, and recent activity panel.

Figure 5.6 shows the feed setup screen used to prepare a local video source before zone tracing and model selection.

Stage Sources

Enter the feed information for one source, optionally attach metadata, then continue through zone tracing, model selection, and review.

One source is prepared at a time. Finish this source, then continue to zone tracing and model selection.

Feed name

Source type

Local MP4

ONVIF Discovery

 **Upload a local video**
Supported formats: MP4, MOV, AVI, and MKV. The backend stores the file and uses its path automatically.

 **Choose Video File**

No video selected yet

Local videos are configured one at a time so each feed keeps a single zone and model.

Step 1 uploads the source file into the backend workflow. Step 2 lets you draw the queue polygon on the video frame. Step 3 selects the model size.

Pick one local video to see it listed here.

Cancel

Continue to Zone

Figure 5.6: Feed setup screen used to prepare a local video source before zone tracing and model selection.

Figure 5.7 shows the zone tracing screen used to define the queue polygon for the selected feed.

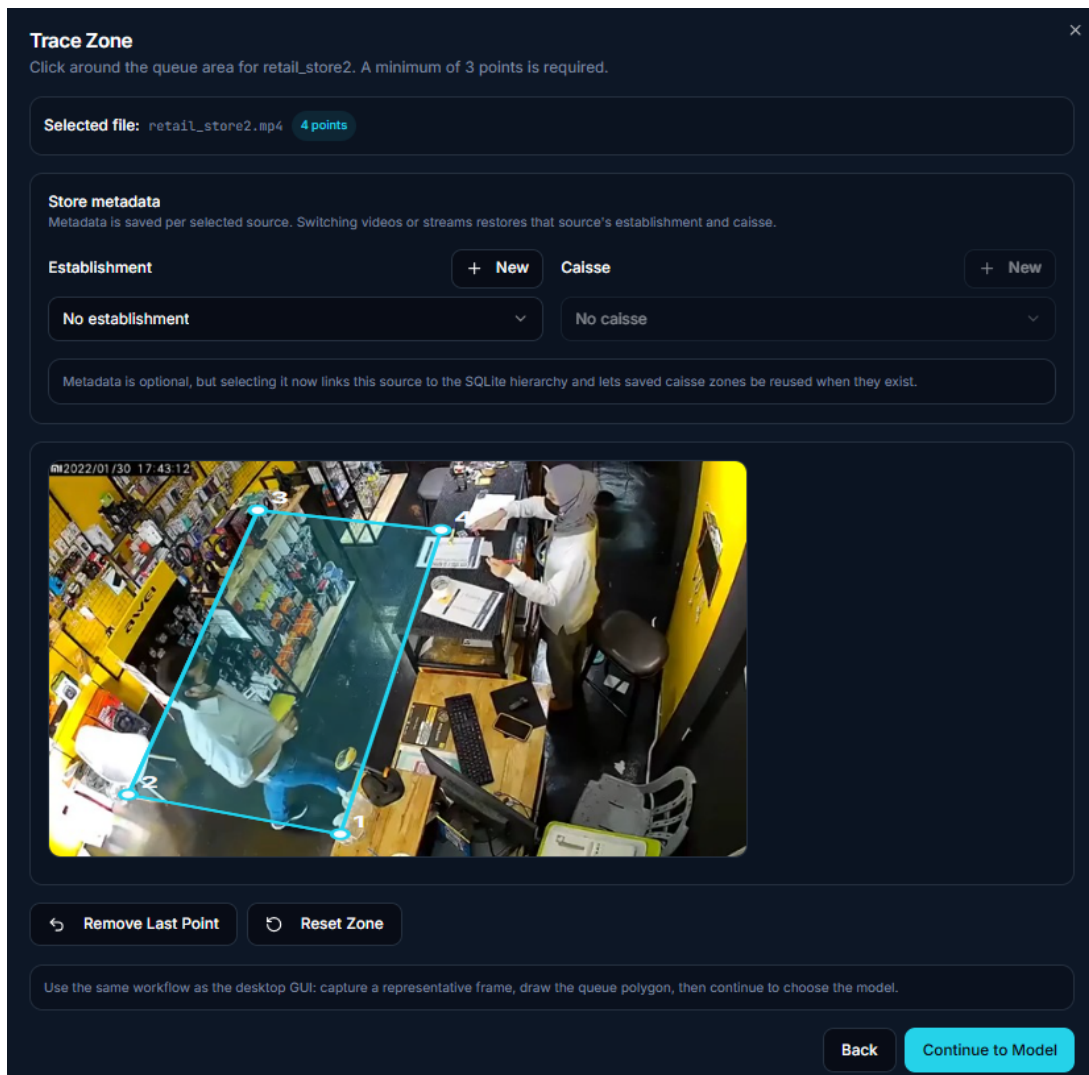


Figure 5.7: Zone tracing screen used to define the queue polygon for the selected feed.

Figure 5.8 shows the model selection screen used to choose the YOLO size before launching the feed.

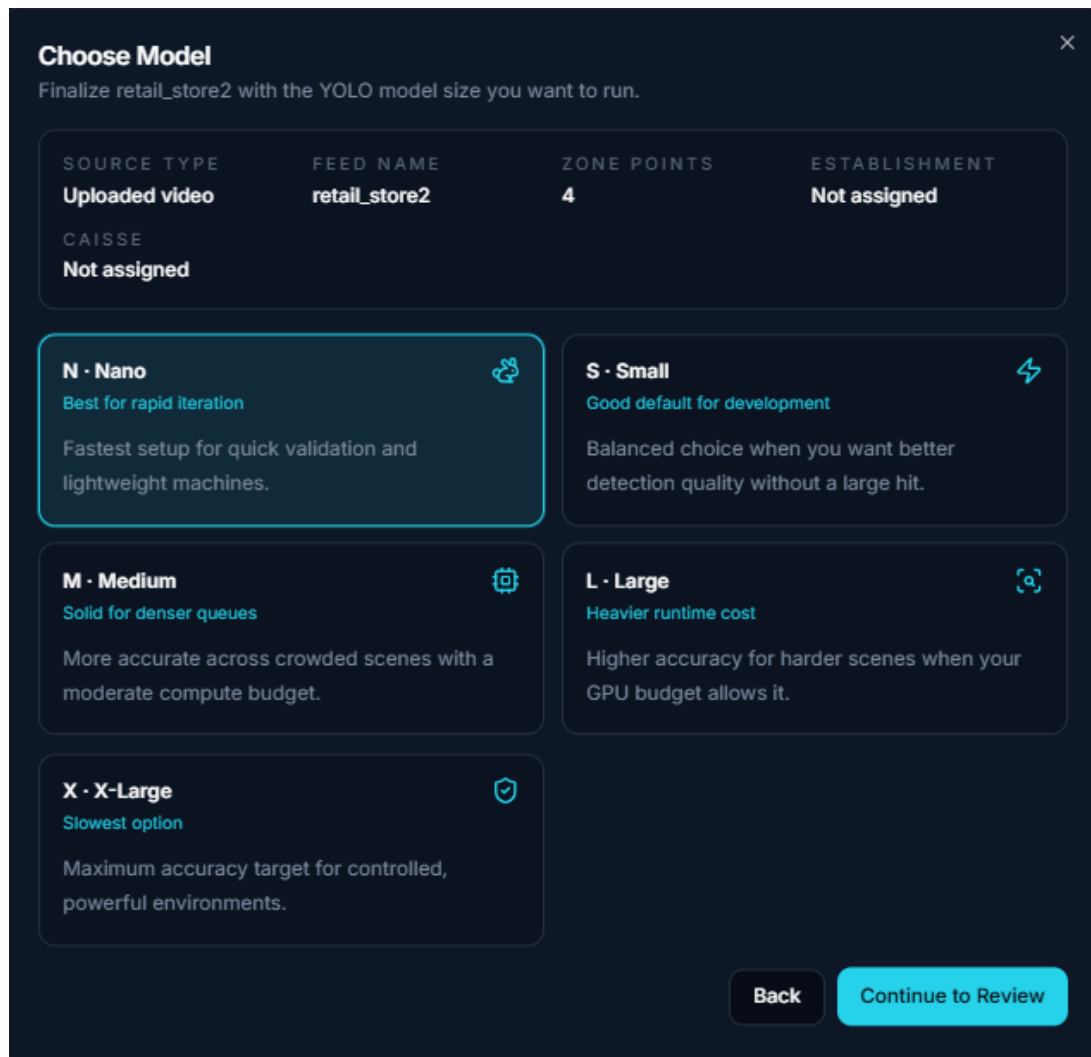


Figure 5.8: Model selection screen used to choose the YOLO size before launching the feed.

Figure 5.9 presents the review and launch panel used to confirm the staged feed before creating the batch.

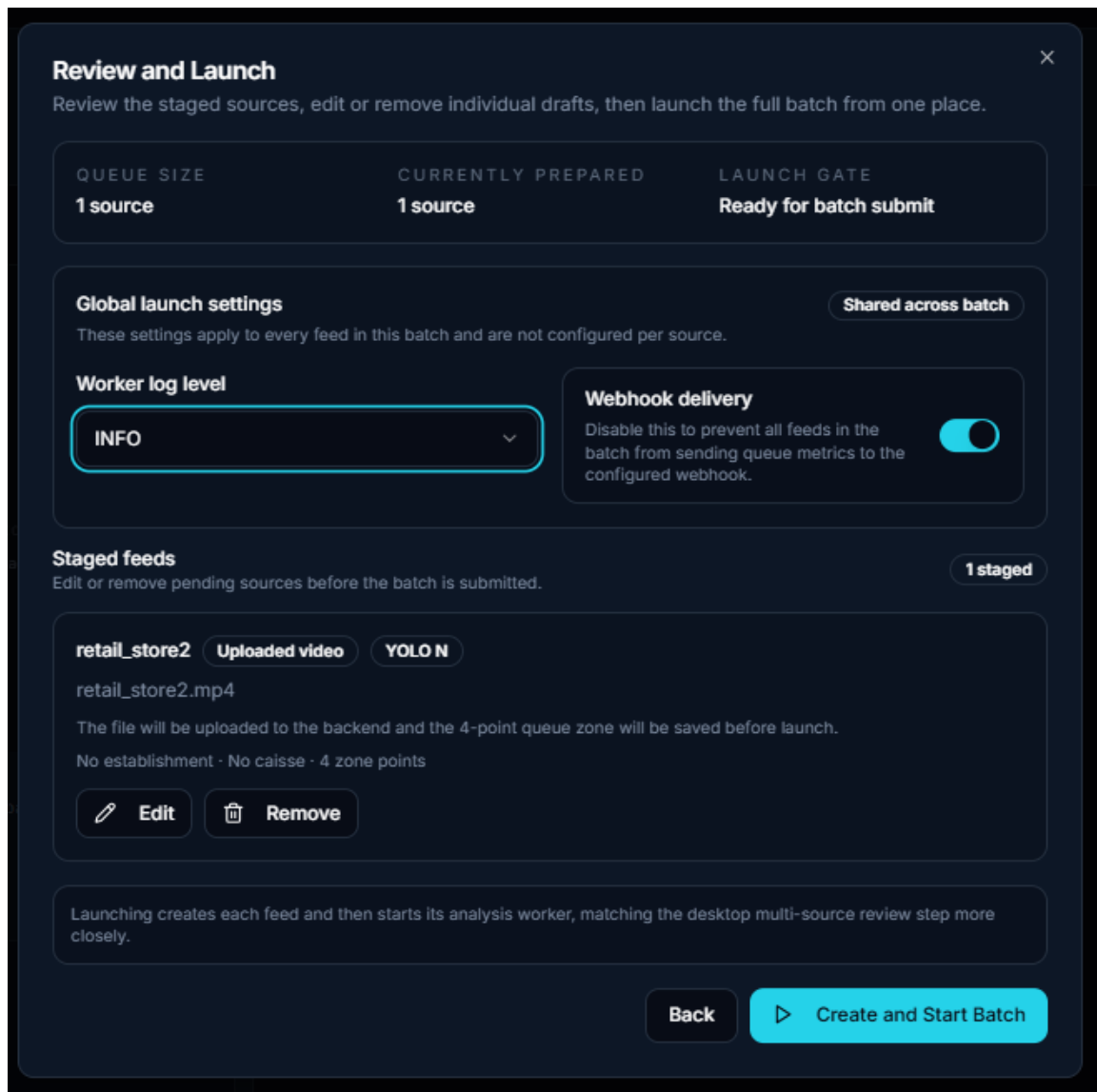


Figure 5.9: Review and launch panel used to confirm the staged feed before creating the batch.

5.5.9 Integration Testing

Frontend integration behavior is validated through scenario-based tests that cover dashboard setup, live event handling, transport fallback, feed routing, and error behavior.

The tests are useful not just for regression detection, but also for documenting what the sprint is supposed to guarantee. They show that the implementation is not finished when the screen renders; it is finished when the system still behaves correctly under reconnects, route changes, and fallback transitions.

5.6 Conclusion

By the end of Sprint 3, the dashboard could react to live state, switch transport modes, and stay usable when the live connection failed. That completed the frontend delivery and prepared the project for alert automation and final evaluation.

5.7 Sprint Retrospective

This sprint confirmed that separating the design foundation from live integration was the right approach. The most useful outcome was the orchestration layer that connects session state, feed actions, live metrics, and transport fallback into one predictable operator experience.

The main tradeoff was complexity at the boundary between frontend and backend. More logic moved into the dashboard, but that logic was necessary to keep the system resilient when transport conditions changed. In the final review, this was considered acceptable because the result is easier to operate and easier to explain in the report.

Sprint 4: Automation, Evaluation, and Optimization

Table of Content

6.1	Introduction	59
6.2	Sprint Objectives	59
6.3	Sprint Backlog	59
6.4	Design	60
6.4.1	Alert Workflow Diagram	60
6.4.2	Verification Plan	61
6.4.3	Performance Targets	61
6.5	Implementation	61
6.5.1	Alert Payload Structure	61
6.5.2	Delivery Reliability	62
6.5.3	Telegram Bot Setup	62
6.5.4	Alert Dispatch in the Runtime Loop	62
6.5.5	n8n Workflow Routing and Telegram Delivery	62
6.5.6	Workflow Validation	63
6.5.7	Verification Coverage	63
6.5.8	Runtime Tuning	63
6.5.9	Fault-Tolerance Improvements	63
6.5.10	Overall Outcome	64
6.6	Conclusion	64

6.7	Sprint Retrospective	64
-----	--------------------------------	----

6.1 Introduction

Sprint 4 closed the project by connecting the queue metrics to external notification workflows and then checking that the full system still held together under realistic conditions. The sprint had two goals: route alerts from the backend through n8n to Telegram so operational staff could be notified automatically, and run a structured evaluation pass to confirm that perception, the API, the dashboard, and the alerting path still worked correctly when exercised together.

This chapter sits at the boundary between analysis and operations. Its role is to keep the runtime pipeline intact while handing alert delivery to a dedicated workflow engine, and then to validate the result.

6.2 Sprint Objectives

This sprint was organized around automation and evaluation outcomes:

Automation objectives:

- define a stable webhook payload and validate it before dispatch,
- add retry-based webhook delivery with logging for failures and successes,
- connect runtime threshold detection to alert triggering,
- configure n8n routing for authorization, cooldown, formatting, and delivery.

Evaluation objectives:

- bring backend and frontend automated tests together into one verification pass,
- confirm feed lifecycle behavior, stream continuity, and API contracts,
- improve runtime throughput and responsiveness,
- strengthen failure handling for network drops, stale data, and worker exits.

6.3 Sprint Backlog

The Sprint 4 backlog covers both the automation tasks and the evaluation work needed to finalize the project. Table 6.1 presents the Sprint 4 backlog.

User StoryID	UserStory	TaskID	Task
US4.1	As a manager, I want alert payloads dispatched to n8n so that operational staff can be notified automatically.	T4.1	Define the webhook payload schema and validate it before dispatch.
US4.2	As a developer, I want webhook delivery to be reliable so that notifications are not silently lost.	T4.2	Add retry logic, timeout handling, and logging for webhook delivery.
US4.3	As a manager, I want alerts routed through Telegram so that I receive notifications on a familiar channel.	T4.3	Create and configure the Telegram bot, store its credential in n8n, and configure the workflow.
US4.4	As a developer, I want the n8n workflow validated with repeatable test scripts.	T4.4	Prepare PowerShell scripts for single-alert, multi-feed, and execution polling scenarios.
US4.5	As a developer, I want the full system verified under realistic conditions so that integration issues are caught before delivery.	T4.5	Run backend and frontend tests together and confirm lifecycle, streaming, and API contracts.
US4.6	As a developer, I want runtime throughput improved so that the processing loop stays responsive.	T4.6	Apply runtime tuning: configurable image size, frame stride, async dispatch, and playback pacing.
US4.7	As a developer, I want fault tolerance strengthened so that the system recovers from transient failures.	T4.7	Improve feed worker supervision, frame handoff continuity, and websocket reconnection.

Table 6.1: Sprint 4 backlog

6.4 Design

6.4.1 Alert Workflow Diagram

Figure 6.1 illustrates the Sprint 4 n8n workflow automation diagram used to route queue alerts through cooldown checks, archival, and Telegram delivery.

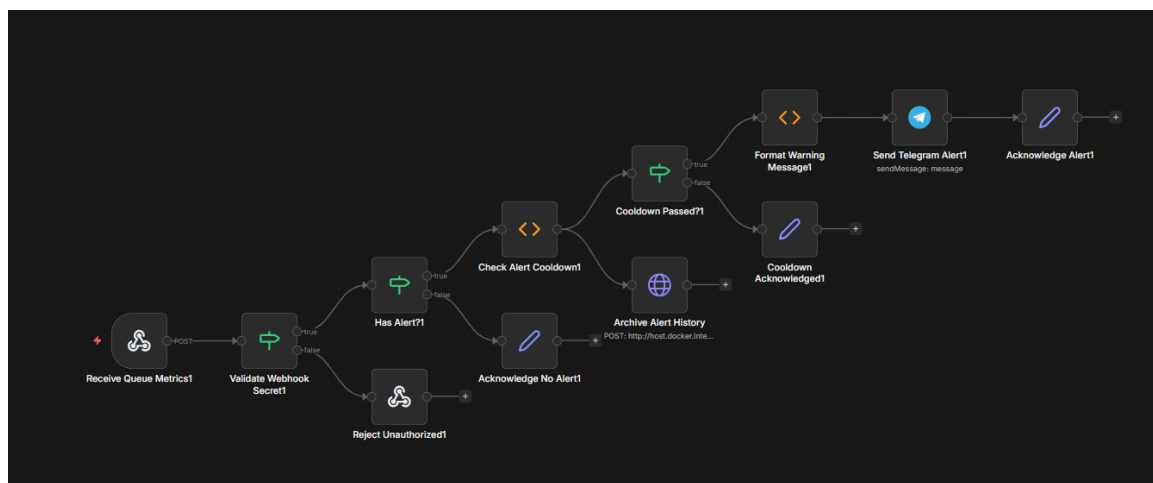


Figure 6.1: Actual Sprint 4 n8n workflow automation diagram used to route queue alerts through cooldown checks, archival, and Telegram delivery.

The automation chain includes the runtime metrics collector, the webhook serializer, the HTTP delivery client, the n8n workflow, and the final notification channel. When an alert needs to be preserved for later review, it goes back to the backend through a dedicated archive endpoint. The workflow checks the webhook secret, applies the cooldown gate, formats the alert, and either acknowledges the payload or forwards it to Telegram.

6.4.2 Verification Plan

Validation followed a layered strategy that covered the system from the backend to the dashboard:

- **Backend unit/integration tests:** queue analytics logic, API endpoints, runtime streaming behavior, and webhook payload handling.
- **Frontend integration tests:** dashboard setup flow, websocket event handling, media fallback behavior, and API error handling.
- **Scenario tests:** n8n send/acknowledge flows, unauthorized access paths, and multi-feed alert scenarios.

6.4.3 Performance Targets

The optimization work was guided by the main operational requirements of the project:

- processing responsiveness, especially loop cadence and processed-frame throughput,
- stream continuity when a feed changes state,
- webhook impact on the main-loop latency,
- dashboard freshness when websocket quality degrades.

These targets intentionally mirror the behavior that an operator would notice first, so the evaluation reads as a system-level check rather than a narrow benchmark exercise.

6.5 Implementation

6.5.1 Alert Payload Structure

The webhook schema is centered on `QueuePayload`, which keeps the information needed by n8n in one structure. It carries:

- source metadata (timestamp, frame, source, feed ID),
- queue metrics (λ , μ , wait time, stability),
- alert flags and severity fields.

Before anything is sent, the payload is validated so malformed records do not leave the backend.

6.5.2 Delivery Reliability

To reduce the risk of missed notifications, the delivery client uses a timeout, a retry policy, an optional shared-secret header, and logging for both successes and failures. Historical payloads remain in SQLite, which gives the system one place to recover past events if needed.

6.5.3 Telegram Bot Setup

Before the workflow could send operational alerts, a Telegram bot was created and connected to the project as the final notification channel. Its API token was added to n8n as a credential instead of being hardcoded, which kept the workflow portable and prevented secret exposure in the exported JSON. Once the credential was available, the Telegram node could use it after the webhook validation and cooldown checks passed.

6.5.4 Alert Dispatch in the Runtime Loop

Inside the main loop, alert sending is moved to `AsyncWebhookDispatcher` so the perception pipeline keeps running without waiting on the network. This makes it possible to send periodic metric updates and immediate alerts, while a deduplication window limits repeated notifications for the same situation.

6.5.5 n8n Workflow Routing and Telegram Delivery

The n8n alert workflow follows a simple path [14]:

- webhook reception and secret validation,
- alert detection and cooldown gating,
- alert history archival via backend API,
- message formatting and Telegram send node,

- explicit acknowledge/reject branches.

Keeping the no-alert, alert-suppressed, and alert-delivered branches separate makes the behavior easier to audit later.

6.5.6 Workflow Validation

Workflow validation was supported by PowerShell scripts for single-alert checks, automated execution polling, and multi-feed scenarios. These scripts made it possible to replay webhook submissions and confirm that each branch behaved as expected during sprint verification.

6.5.7 Verification Coverage

The test suite covers API transport behavior, runtime stream handling, webhook delivery, dashboard integration, live-state synchronization, API error cases, and transport fallback behavior. Taken together, these tests check the API envelope, the event contract, stream handoff, and the setup workflow.

6.5.8 Runtime Tuning

The main tuning actions applied across the runtime loop and configuration layers included:

- configurable detector image size and frame stride,
- explicit control over the inference device and its diagnostics,
- asynchronous webhook dispatch so the processing loop stayed responsive,
- real-time playback pacing for file-based sources.

6.5.9 Fault-Tolerance Improvements

Runtime resilience was improved at several levels:

- feed worker supervision and exit recovery in `FeedRegistry`,
- capture-to-worker frame handoff continuity in `FeedFrameStreamManager`,
- websocket reconnect and fallback behavior on the frontend side.

6.5.10 Overall Outcome

By the end of the sprint, the system had reached a stable integration point where perception, analytics, the API runtime, the dashboard, and n8n automation operated as one pipeline. What remains is mainly deployment hardening and longer-term benchmarking, not missing core functionality.

6.6 Conclusion

Sprint 4 closed the project with operational alert routing and structured testing. The automation path now delivers alerts from the backend to Telegram through n8n, while the evaluation pass confirmed that the full stack still works under realistic conditions. At this stage, the platform is ready for demonstration and academic evaluation, while the remaining work is mainly deployment hardening and longer-term benchmarking.

6.7 Sprint Retrospective

This final sprint showed that the most important delivery was not any individual feature but the end-to-end integration. The automation work gave the system its last operational channel, and the evaluation work confirmed that the earlier sprints still held up when tested together.

The main lesson is that alerting and verification are not secondary tasks. Without reliable dispatch and structured testing, the earlier perception and dashboard work would remain incomplete from an operational perspective.

Table of Content

7.1	Summary	66
7.2	Objective Review	66
7.3	Future Work	67

7.1 Summary

This end-of-studies project produced a complete queue wait-time estimation platform that brings together computer vision, queue analytics, web supervision, and automation workflows.

In practical terms, the system now includes:

- a perception pipeline based on YOLO detection and ByteTrack identity continuity,
- queue-zone filtering and event-driven estimation of λ , μ , and expected wait time,
- stability-aware outputs with explicit queue-state semantics,
- a secure authentication layer for manager sign-up, sign-in, session refresh, and administrator account governance,
- a typed FastAPI backend-for-frontend with feed lifecycle control and live websocket events,
- a React dashboard for monitoring, setup, and runtime interaction,
- n8n and Telegram integration for operational alert routing.

The resulting architecture shows that a full monitoring-to-action loop can be built from modular components without losing maintainability or testability.

Across the four consolidated sprint chapters, each sprint adds one layer to that loop: Sprint 1 establishes perception and queue analytics, Sprint 2 builds the backend services and API integration, Sprint 3 delivers the web dashboard and live monitoring, and Sprint 4 completes the automation pipeline and validates the full system.

7.2 Objective Review

The initial objectives were achieved at the project scope level:

1. **Real-time detection and tracking:** achieved through dedicated detector and tracker modules.
2. **Queue-zone counting:** achieved with polygon-zone management and in-zone identity filtering.
3. **Queue metric estimation:** achieved for arrival rate, service rate, and wait time.
4. **Queue stability handling:** achieved through stable and unstable classification with fallback wait-time logic.
5. **Web monitoring:** achieved through a multi-feed dashboard with live updates.

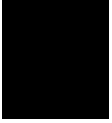
6. **Automation integration:** achieved through webhook transport, n8n routing, and Telegram notifications.

The main remaining gap is not architectural completeness, but the depth of long-horizon evaluation under broader operational scenarios.

7.3 Future Work

Possible extensions include:

- multi-zone and multi-server queue modeling for parallel cashier configurations,
- richer alert strategies with cooldown tuning, escalation policies, and acknowledgment loops,
- edge deployment optimization through ONNX/TensorRT profiling and resource-aware scheduling,
- long-duration field evaluation with domain-specific KPI baselines,
- stronger security controls for production webhook exposure and secrets management.



References

- [1] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Wiley, 2018.
- [2] Ultralytics, *Ultralytics documentation*, <https://docs.ultralytics.com/>, 2026.
- [3] Waitwhile, *Waitwhile – virtual queue and appointment management*, <https://www.waitwhile.com/>, Accessed: 2026-04-20, 2026.
- [4] Qminder, *Qminder – queue management system*, <https://www.qminder.com/>, Accessed: 2026-04-20, 2026.
- [5] RetailNext, *Retailnext – in-store analytics and traffic intelligence*, <https://retailnext.net/>, Accessed: 2026-04-20, 2026.
- [6] Xovis, *Xovis – 3d sensor-based people flow and queue measurement*, <https://www.xovis.com/>, Accessed: 2026-04-20, 2026.
- [7] Ultralytics, *Yolo by ultralytics – real-time object detection*, <https://docs.ultralytics.com/models/>, Accessed: 2026-04-20, 2026.
- [8] Roboflow, *Queue monitoring with computer vision*, <https://roboflow.com/use-cases/queue-monitoring>, Accessed: 2026-04-20, 2026.
- [9] Y. Zhang et al., “Bytetrack: Multi-object tracking by associating every detection box,” *arXiv preprint arXiv:2110.06864*, 2022.
- [10] OpenCV, *Opencv documentation*, <https://docs.opencv.org/>, 2026.
- [11] W3C Web Real-Time Communications Working Group, *Webrtc 1.0: Real-time communication between browsers*, <https://www.w3.org/TR/webrtc/>, Accessed: 2026-04-07, 2025.

REFERENCES

- [12] MediaMTX, *Mediamtx documentation*, <https://mediamtx.org/docs/>, Accessed: 2026-04-07, 2026.
- [13] Internet Engineering Task Force, *Rfc 8830: WebRTC media transport and use of RTP*, <https://www.rfc-editor.org/rfc/rfc8830>, Accessed: 2026-04-07, 2021.
- [14] n8n, *N8n documentation – workflow automation*, <https://docs.n8n.io/>, Accessed: 2026-04-20, 2026.